

FINAL YEAR PROJECT REPORT

Project title:

**Design and Development of a Real-Time Queue Monitoring and
Wait-Time Estimation System**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

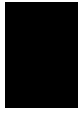
Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	viii
List of Tables	x
List of Abbreviations and Symbols	xi
General Introduction	1
1 Context of the Project	2
1.1 Introduction	3
1.2 Host Organization	3
1.2.1 DataDoIt	3
1.2.2 Sectors of Activity	4
1.3 Presentation of the Project	5
1.3.1 Project Context	5
1.3.2 Problem Statement	5
1.3.3 Study of Existing Approaches	6
1.3.3.1 Queue Management Platforms	6
1.3.3.2 Sensor-Based Analytics	7
1.3.3.3 Vision-Based Monitoring Frameworks	7
1.3.3.4 Positioning of the Proposed Solution	7
1.3.4 Proposed Solution and Project Objectives	8
1.3.5 Project Scope, Boundaries, and Stakeholders	9
1.4 Project Management Methodology	10
1.4.1 Choice of Methodology: Agile	10

1.4.2	SCRUM	10
1.4.3	Sprint Planning and Execution	11
1.5	Conclusion	12
2	Requirements Specification and Analysis	13
2.1	Introduction	14
2.2	Requirements Specification	14
2.2.1	Identification of Actors	14
2.2.2	Functional Requirements	15
2.2.3	Non-functional Requirements	17
2.3	Product Backlog and Sprint Organization	17
2.4	Global Use Case Diagram	18
2.5	Project Environment	19
2.5.1	Hardware and Software Environment	19
2.5.2	Development Tools	19
2.5.3	Technologies Used	20
2.6	Software Architecture	21
2.6.1	Overall Architecture	21
2.6.2	Backend Architecture	23
2.6.3	Frontend Architecture	23
2.7	Conclusion	24
3	Sprint 1: Perception and Queue Analytics	25
3.1	Introduction	26
3.2	Sprint Objectives and Backlog	26
3.3	Sprint Backlog	26
3.4	Design	28
3.4.1	Perception and Analytics Flow	28

3.5	Implementation	31
3.5.1	Perception Pipeline	31
3.5.2	Queue Event Windows and Rate Estimation	31
3.5.3	Wait-Time Logic and Adaptive Estimation	32
3.5.4	Runtime Metric Contract	32
3.6	Conclusion	33
4	Sprint 2: Backend Services and API Integration	34
4.1	Introduction	35
4.2	Sprint Objectives and Backlog	35
4.3	Design	36
4.3.1	Backend Domain Model and Class Structure	36
4.4	Implementation	37
4.4.1	Source Integration and Camera Discovery	37
4.4.2	API and Live Transport	38
4.4.3	Persistence, State Recovery, and System Resilience	41
4.5	Conclusion	41
5	Sprint 3: Web Dashboard and Live Monitoring	42
5.1	Introduction	44
5.2	Sprint Objectives and Backlog	44
5.2.1	User Stories	44
5.2.2	Technical Tasks	46
5.3	Design	46
5.3.1	Visual Design and Interface Architecture	46
5.3.2	Dashboard Data Flow and Integration Architecture	47
5.3.3	Design Rationale	47
5.4	Implementation	47

5.4.1	Project Setup and Layout Skeleton	47
5.4.2	Styling and Reusable Interface Rules	49
5.4.3	API Integration	49
5.4.4	Authentication and Role-Based Navigation	49
5.4.5	Dashboard State Orchestration and Live Update Reliability	50
5.4.6	Live Video Delivery and Transport Selection	52
5.4.7	Dashboard UI Integration	52
5.4.8	Manager Onboarding Workflow	52
5.4.9	Analytics and Historical Reporting	53
5.4.10	Integration Testing	55
5.5	Conclusion	56
6	Sprint 4: Automation, Evaluation, and Optimization	59
6.1	Introduction	60
6.2	Sprint Objectives and Backlog	60
6.2.1	Technical Tasks	61
6.3	Design	62
6.3.1	Alert Automation and n8n Workflow	62
6.3.2	System Verification and Evaluation Strategy	62
6.4	Implementation	63
6.4.1	Alert Payload and Delivery Architecture	63
6.4.2	Telegram Integration and Notification Delivery	63
6.4.3	Asynchronous Alert Dispatch and Deduplication	65
6.4.4	n8n Workflow Structure and Validation	66
6.4.5	Comprehensive Integration Verification	66
6.4.6	Runtime Optimization and Performance Tuning	67
6.4.7	Fault-Tolerance and Recovery Mechanisms	67

6.4.8	System Integration and Operational Readiness	67
6.5	Conclusion	67
7	General Conclusion	69
7.1	Summary	70
7.2	Objective Review	71
7.3	Future Work	71



List of Figures

1.1	DataDoIt company logo	4
1.2	Scrum process used to organize the project.	10
2.1	Global use case diagram of the queue wait-time estimation system. . . .	18
2.2	Overall system architecture from video acquisition to alert delivery. . . .	22
3.1	Sprint 1 activity flow from frame ingestion to queue metrics output. . . .	29
3.2	Sprint 1 analytics activity diagram for queue metric generation.	30
4.1	Sprint 2 backend class diagram showing core entities for feed orchestra- tion and persistence.	37
4.2	Camera discovery activity diagram: ONVIF discovery, RTSP resolution, and stream validation.	39
4.3	ONVIF protocol sequence: WS-Discovery broadcast, direct media service resolution, and device service indirection.	40
5.1	Dashboard data flow sequence showing protected access (alt), metric broadcasts (loop), fallback polling (opt), and video transport negotiation (alt).	48
5.2	Manager sign-in screen for dashboard access.	50
5.3	Manager account creation screen.	51
5.4	Administrator manager-administration interface.	51
5.5	Manager dashboard: live surface, feed summaries, and recent activity. .	53

5.6	Manager onboarding activity diagram showing user actions and UI state transitions from feed setup initialization through feed launch confirmation. Decision nodes show ONVIF versus upload path selection and the test validation retry loop.	54
5.7	Feed setup screen used to prepare an uploaded video or ONVIF camera source before zone tracing and model selection.	55
5.8	Zone tracing screen used to define the queue polygon for the selected feed.	56
5.9	Model selection screen used to choose the YOLO size before launching the feed.	57
5.10	Review and launch panel used to confirm staged feeds before batch creation or launch.	58
5.11	Analytics dashboard: KPI summaries, trends, zone rankings, and export.	58
6.1	Sprint 4 n8n workflow for routing queue alerts through cooldown checks, archival, and Telegram delivery.	62
6.2	Sprint 4 evaluation and resilience activity diagram for tests, tuning, and recovery verification.	64
6.3	Example Telegram alert delivered.	65



List of Tables

1.1	From operational pain points to system-level requirements	6
1.2	Positioning of the proposed solution against existing approaches	8
1.3	Objective-to-module traceability	9
1.4	Scrum module mapping used in this report	11
1.5	Sprint planning used in the report	12
2.1	Functional requirement traceability matrix	16
2.2	Non-functional requirement quality matrix	17
2.3	Primary global use cases	19
2.4	Hardware environment	19
2.5	Development tools used in the project	20
2.6	Technologies used in the project	21
3.1	Sprint 1 user stories	27
3.2	Sprint 1 technical tasks	28
4.1	Sprint 2 user stories	36
5.1	Sprint 3 user stories	45
5.2	Sprint 3 technical tasks	46
6.1	Sprint 4 user stories	60
6.2	Sprint 4 technical tasks	61



List of Abbreviations and Symbols

The following symbols and abbreviations are used throughout the report.

Symbols

λ	Arrival rate.
μ	Service rate.
W	Expected wait time.
L	Number of people currently in the queue zone.

Abbreviations

API

Application Programming Interface.

BFF

Backend for Frontend.

CSV

Comma-Separated Values.

HTTP

Hypertext Transfer Protocol.

MJPEG

Motion JPEG.

ONVIF

Open Network Video Interface Forum.

REST

Representational State Transfer.

RTSP

Real-Time Streaming Protocol.

TCP

Transmission Control Protocol.

UDP

User Datagram Protocol.

UI

User Interface.

WebRTC

Web Real-Time Communication.

WHEP

WebRTC HTTP Egress Protocol.

WS-Discovery

Web Services Discovery.



General Introduction

In many service environments, waiting time is one of the first signs that the organization is under pressure. Supermarkets, banks, and service centers may have enough staff during normal periods, but a sudden increase in demand can quickly create queues that are difficult to evaluate by observation alone.

In practice, customers usually join a queue without knowing how long they will wait. Managers and operational staff also have limited real-time information for deciding when to open another service point, redirect customers, or escalate the situation. Manual observation remains subjective, and simple counting tools do not explain how fast the queue is moving or whether the waiting time is becoming unstable.

This end-of-studies project addresses that gap through a real-time queue monitoring system based on computer vision and queueing theory. The system detects and tracks people in video streams, restricts the analysis to configurable queue zones, estimates arrival and service rates, computes waiting time, and presents the resulting indicators in a form that can support operational decisions.

It combines classical queueing theory with computer-vision detection and tracking techniques [1], [2] to provide actionable waiting-time estimates.

The report is organized as follows. Chapter 1 presents the project context, the host organization, the problem statement, and the objectives. Chapter 2 defines the requirements and the main architecture choices. Chapters 3 to 6 describe the work sprint by sprint, from perception and queue analytics to the web dashboard and alert automation. The general conclusion closes the report and presents future work directions.

Context of the Project

Table of Content

1.1	Introduction	3
1.2	Host Organization	3
1.2.1	DataDoIt	3
1.2.2	Sectors of Activity	4
1.3	Presentation of the Project	5
1.3.1	Project Context	5
1.3.2	Problem Statement	5
1.3.3	Study of Existing Approaches	6
1.3.4	Proposed Solution and Project Objectives	8
1.3.5	Project Scope, Boundaries, and Stakeholders	9
1.4	Project Management Methodology	10
1.4.1	Choice of Methodology: Agile	10
1.4.2	SCRUM	10
1.4.3	Sprint Planning and Execution	11
1.5	Conclusion	12

1.1 Introduction

Queue monitoring is useful only when it connects the observed situation to a decision. For this reason, the project was not limited to counting people in a scene; it had to connect visual observation, waiting-time estimation, dashboard supervision, and alerting.

This chapter presents the setting in which the project was carried out: the collaboration with DataDoIt, the operational problem, the comparison with existing approaches, the solution scope, and the Scrum organization used during development.

1.2 Host Organization

1.2.1 DataDoIt

This end-of-studies project was carried out with DataDoIt, a data-driven sustainability consulting firm headquartered at Elghazal Technopark in Tunis, Tunisia. DataDoIt works on projects that connect data science and sustainability for governments, municipalities, and enterprises across the EMEA region. Its services cover decarbonization strategy, ESG reporting, smart resource analytics, and circular economy implementation. The company uses the motto Measure It, Optimize It, Do It to describe a working method based on analysis, planning, and deployment. At the time of the project, it had delivered more than 15 active projects across 6 sectors, with a reported average ROI of 2.3x and a 94% client retention rate.

The collaboration gave the project a concrete setting: collect useful data from live environments, transform it into indicators, and make those indicators available quickly enough for action. These concerns shaped the design and scope of the system. The DataDoIt company logo is shown in Figure 1.1.

The collaboration perimeter centered on queue management in service scenarios,



Figure 1.1: DataDoIt company logo

multi-source video ingestion, low-latency monitoring, actionable queue indicators, and iterative validation across the main system layers.

1.2.2 Sectors of Activity

DataDoIt delivers sustainability solutions across six mission-critical sectors where data-driven environmental performance has direct operational and financial implications:

- government and public sector institutions seeking to translate climate policy into measurable action,
- smart cities and municipalities requiring integrated urban resilience and climate adaptation systems,
- industrial and manufacturing operations targeting Scope 1-3 carbon footprint reduction and supply chain efficiency,
- real estate and infrastructure organizations pursuing green certification and climate-resilient asset development,
- financial services firms navigating ESG compliance and green finance opportunities,
- energy and utilities providers accelerating the transition toward renewable integration and grid modernization.

1.3 Presentation of the Project

1.3.1 Project Context

Manual queue supervision is subjective, hard to scale, and dependent on continuous attention. In many settings, teams react only after congestion is already visible, because they do not have a measured signal that warns them earlier.

This project addresses that gap by turning camera streams into decision-oriented telemetry. The target output includes people in zone, arrival rate, service rate, adaptive wait-time estimates, and alert states for monitoring and automation layers.

1.3.2 Problem Statement

Current operational practices and basic counting tools still have recurring limitations:

- they provide limited anticipation of congestion,
- they rarely estimate waiting time explicitly,
- they provide weak visibility into queue stability during sudden demand spikes,
- they are often disconnected from automated response workflows.

As a result, interventions such as staff reallocation, lane opening, or escalation can be delayed or triggered from incomplete information. The problem is therefore both analytical and practical: the system must estimate queue behavior and deliver the result to the people who can act on it. Table 1.1 maps these pain points to system-level requirements.

Observed pain point	Operational impact	Required system capability
Manual visual estimation is inconsistent across shifts.	Queue state interpretation varies between staff members, reducing decision reliability.	Standardized per-feed metrics with explicit formulas and stable update cadence.
Occupancy-only monitoring is descriptive but not predictive.	Teams react late because no forward-looking wait-time signal is available.	Continuous estimation of λ , μ , and expected wait time W .
Queue behavior under load spikes is hard to interpret.	Teams may overreact or underreact when service and arrivals fluctuate quickly.	Adaptive wait-time formula that responds gracefully to changing queue conditions.
Alerting tools are not integrated with runtime analytics.	Escalation depends on manual relay, increasing response latency.	Automated webhook dispatch and workflow-based notification routing.

Table 1.1: From operational pain points to system-level requirements

1.3.3 Study of Existing Approaches

Several computer-vision frameworks provide the basic building blocks for person detection and tracking. In parallel, queueing theory gives a formal way to describe service-system behavior and relate arrival rate, service rate, and expected waiting time [1].

In practice, existing commercial and open-source tools cover only parts of the queue-management problem. They fall into three broad families: queue management platforms, sensor-based analytics, and vision-based monitoring frameworks.

1.3.3.1 Queue Management Platforms

Platforms such as Waitwhile [3] and Qminder [4] focus on active queue management. They let customers join virtual queues through kiosks, QR codes, or web links, and they notify staff through SMS or dashboards. Their strength is customer-facing experience: reduced perceived waiting, appointment scheduling, and service routing. However,

they require explicit customer interaction (check-in) and do not observe the physical queue directly. They cannot estimate wait time from the scene itself, and they do not detect congestion unless a customer has already registered.

1.3.3.2 Sensor-Based Analytics

RetailNext [5] and Xovis [6] represent the sensor-based analytics family. RetailNext uses ceiling-mounted AI sensors to track foot traffic, measure conversion rates, and optimize staffing in retail environments. Xovis uses 3D optical sensors for precise people flow and queue length measurement, particularly in airports and large retail spaces. Both provide high-accuracy occupancy and dwell-time data, but they require proprietary hardware installations and are primarily designed for large-scale enterprise deployments. They observe the queue passively but do not typically integrate with external automation workflows or provide open APIs for custom alert routing.

1.3.3.3 Vision-Based Monitoring Frameworks

Open-source tools built on YOLO detection and tracking, such as those provided by Ultralytics [2] and Roboflow [7], offer the computer-vision building blocks needed for queue monitoring: person detection, tracking, and zone-based counting. These frameworks are flexible and can be adapted to specific use cases. However, they typically stop at detection and counting. They do not include queue-theoretic estimation of wait time, stability classification, or integration with notification workflows. The gap between raw detection output and actionable operational metrics is left to the developer.

1.3.3.4 Positioning of the Proposed Solution

The proposed solution is positioned against the main existing approach families in Table 1.2.

Approach family	Strengths	Typical limitation
Queue management platforms (Waitwhile, Qminder)	Customer-facing experience, virtual queues, appointment scheduling.	Require explicit check-in; do not observe physical queues or estimate wait time from the scene.
Sensor-based analytics (RetailNext, Xovis)	High-accuracy occupancy and flow data from dedicated 3D sensors.	Proprietary hardware; limited custom automation; enterprise-scale pricing.
Vision-based detection frameworks (Ultralytics, Roboflow)	Flexible open-source detection and tracking building blocks.	Stop at counting; no queue-theoretic estimation, stability classification, or alert integration.
Manual or occupancy-only monitoring	Easy to deploy and interpret.	Weak predictive value and high subjectivity.
Proposed integrated pipeline	Links perception, queue analytics, stability handling, web monitoring, and automation in one modular architecture.	Requires careful integration and validation across multiple technical domains.

Table 1.2: Positioning of the proposed solution against existing approaches

The proposed solution combines open-source vision-based detection with queue-theoretic analytics, a web monitoring dashboard, and automated alert routing in one modular pipeline without proprietary hardware or explicit customer interaction.

1.3.4 Proposed Solution and Project Objectives

The proposed system links perception, analytics, monitoring, and automation in a modular architecture. Instead of stopping at raw visual observations, it maps camera feeds to queue-state information, then makes that information available through dashboard views and alert workflows.

To establish clear traceability between the technical solution and measurable project

outcomes, the objectives are mapped to delivery modules and observable outputs in Table 1.3.

Objective ID	Objective	Primary module	Observable output
OBJ-01	Real-time detection and tracking	Module A	Stable in-frame detections with persistent tracker IDs.
OBJ-02	Queue-zone counting	Module A	Per-frame in-zone people count and zone-aware overlays.
OBJ-03	Queue metric estimation (λ, μ, W)	Module A	Continuous metric payloads from queue analysis pipeline.
OBJ-04	Adaptive wait-time estimation	Module A	Compute wait time with adaptive logic that responds to queue load.
OBJ-05	Web monitoring	Module B	Live dashboard with feed controls, KPIs, and stream views.
OBJ-06	Automation integration	Module C	Webhook-based alert routing through n8n and Telegram.

Table 1.3: Objective-to-module traceability

1.3.5 Project Scope, Boundaries, and Stakeholders

The implementation scope covers three distinct but interdependent domains: perception and queue analytics, runtime services and integration, and web-based operational interfaces. The report follows the same structure so that design decisions and validation activities remain easy to trace.

The system is designed for three stakeholder groups with complementary responsibilities. Administrators manage account governance and access controls. Managers configure feeds and thresholds, monitor live queue status, respond to alerts, and review performance indicators. End users benefit indirectly through shorter waiting times and smoother service flow. This separation keeps each stakeholder focused on the part of the system they actually use.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

An Agile approach was selected because the project combined several domains that had to work together: computer vision, queue analytics, runtime services, frontend workflows, and automation. Building all of them in a single delivery cycle would have made integration risks harder to detect.

Iterative delivery made it possible to validate the project progressively: first the perception output, then the queue metrics, then the live dashboard behavior, and finally the alert workflow and system hardening.

1.4.2 SCRUM

Development followed a sprint-based Scrum methodology with validation at the end of each major increment. Each sprint focused on a coherent part of the product: perception and analytics first, then camera connectivity and live services, then the web dashboard, and finally alert automation and evaluation. For readability and traceability, the work is presented through three logical modules, as illustrated in Figure 1.2.

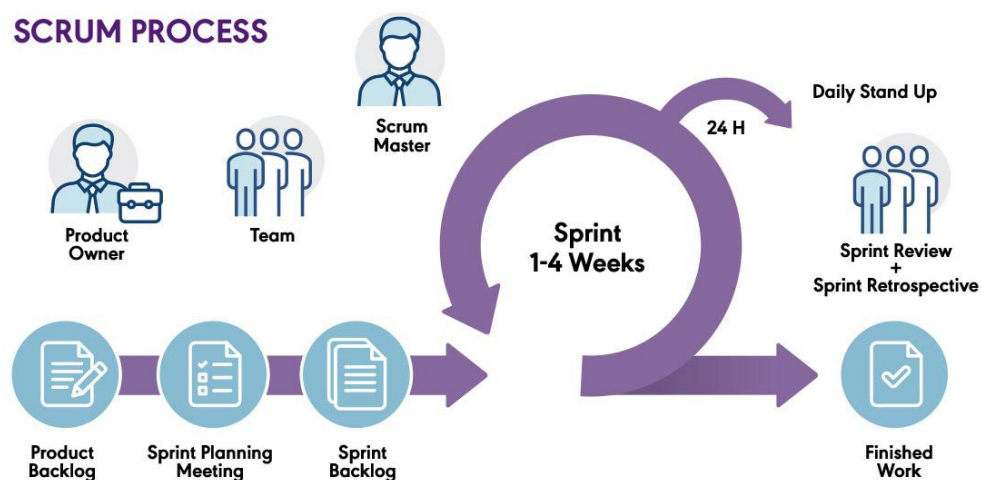


Figure 1.2: Scrum process used to organize the project.

The modular organization is summarized in Table 1.4, which maps each module to its constituent sprints, primary focus areas, and validation perspectives.

Module	Sprints	Main focus	Validation perspective
Module A: Backend Core	1 to 2	Perception pipeline, queue analytics, camera connectivity, API services, live transport	Perception correctness, metric contracts, and API reliability.
Module B: Web Experience	3	Frontend design system, dashboard scaffolding, live integration, monitoring workflows	Usability, live-rendering stability, and role-based navigation.
Module C: Automation and Finalization	4	n8n alerting, Telegram notifications, evaluation, optimization, and fault tolerance	End-to-end operability and delivery readiness.

Table 1.4: Scrum module mapping used in this report

1.4.3 Sprint Planning and Execution

The project was structured into four sprints, each with defined objectives, deliverables, and validation checkpoints. The sprint plan is summarized in Table 1.5, which details the timeline, focus areas, and traceability to user story implementations.

Sprint	Objectives	Start Date	End Date	User Story IDs
Sprint 1	Perception and queue analytics, detection, tracking, and zone filtering, wait-time estimation and stability handling	16/02/2026	14/03/2026	US1.1–US1.9
Sprint 2	Camera connectivity and discovery, backend API exposure, live transport and feed management	15/03/2026	11/04/2026	US2.1–US2.4
Sprint 3	Web design system and frontend scaffolding, live backend integration, dashboard monitoring and role-based navigation	12/04/2026	09/05/2026	US3.1–US3.9
Sprint 4	n8n automation and Telegram alerting, testing and verification, runtime tuning and fault tolerance	10/05/2026	06/06/2026	US4.1–US4.7

Table 1.5: Sprint planning used in the report

The detailed user stories for each sprint are presented in the backlog tables in Chapters 3 to 6.

1.5 Conclusion

This chapter defined the project context and the delivery strategy. Chapter 2 builds on this baseline by presenting the requirements, architecture constraints, and analysis artifacts used to guide the implementation.

Table of Content

2.1	Introduction	14
2.2	Requirements Specification	14
2.2.1	Identification of Actors	14
2.2.2	Functional Requirements	15
2.2.3	Non-functional Requirements	17
2.3	Product Backlog and Sprint Organization	17
2.4	Global Use Case Diagram	18
2.5	Project Environment	19
2.5.1	Hardware and Software Environment	19
2.5.2	Development Tools	19
2.5.3	Technologies Used	20
2.6	Software Architecture	21
2.6.1	Overall Architecture	21
2.6.2	Backend Architecture	23
2.6.3	Frontend Architecture	23
2.7	Conclusion	24

2.1 Introduction

After defining the project context, the next step was to translate the idea into requirements that could guide the work. This chapter identifies the actors, lists the expected functions and quality constraints, and shows how those requirements are distributed across the implementation plan.

The requirements follow the same queue-monitoring path used throughout the project: video ingestion, person detection and tracking, queue-metric estimation, dashboard delivery, authentication, and alert automation.

2.2 Requirements Specification

2.2.1 Identification of Actors

The platform involves two actors inside the application and one operational actor outside it. This distinction matters because the software can detect, estimate, and notify, but the physical response to congestion still happens on site.

The actors are:

- **Administrator:** manages manager accounts (create, update, delete) and controls access governance.
- **Manager:** configures sources and thresholds, monitors live queues, reacts to alerts, and reviews performance indicators.
- **On-site Operator (external actor):** receives queue alert notifications in a shared Telegram group and applies field actions, such as opening a new queue line or redirecting customer flow. In the MVP, the operator confirms completion manually in Telegram, and this confirmation is not automatically captured by the application.

This split keeps the functional scope clear: access control, monitoring, and alert notification remain inside the system, while physical queue intervention and manual completion confirmation are treated as external operational actions.

2.2.2 Functional Requirements

At a high level, the system captures a feed, counts people inside a queue zone, estimates queue behavior, updates the dashboard, and sends alerts when the configured conditions are reached. These requirements define the minimum end-to-end behavior expected from the platform and are formalized in Table 2.1.

ID	Requirement	Priority	Verification
FR-01	Detect people in live or recorded streams in near real time.	High	Validate detections and overlays in the perception layer.
FR-02	Preserve identity continuity across frames for queue analysis.	High	Validate tracker ID continuity in runtime scenarios.
FR-03	Restrict counting to configurable polygon queue zones.	High	Validate polygon inclusion in the zone-management layer.
FR-04	Compute λ , μ , and expected wait time from event windows.	High	Validate queue formulas and stability handling in analytics.
FR-05	Compute wait time with adaptive logic that switches between precise and conservative estimates.	High	Validate formula branches and fallback behavior.
FR-06	Expose feed control and live updates through API and WebSocket channels.	High	Validate endpoint and runtime behavior in API tests.
FR-07	Dispatch alert payloads to automation workflows.	Medium	Validate webhook dispatch and n8n branching.
FR-08	Persist operational metrics and metadata for analysis and auditing.	Medium	Validate persistence behavior in runtime and database checks.
FR-09	Allow manager registration, sign-in, refresh, and logout through secure cookies.	High	Validate register, login, refresh, logout, and session checks.
FR-10	Allow administrator to create, update, deactivate, and reset manager accounts.	High	Validate account-governance endpoints and audit events.
FR-11	Enforce password policy, logout after repeated failures, and role-based protection on operational endpoints.	High	Validate failed-login counters, logout behavior, and unauthorized access.
FR-12	Record authentication and account-governance actions for traceability.	Medium	Validate audit-log entries during register and login flows.
FR-13	Protect administrative and operational APIs with role-based access control.	High	Validate unauthorized access paths and audit events.

Table 2.1: Functional requirement traceability matrix

2.2.3 Non-functional Requirements

In addition to functional behavior, the platform must satisfy quality constraints that keep it usable in a live monitoring environment. A correct wait-time formula is not enough if the video feed freezes, the dashboard stops updating, or alerts are lost. For this reason, performance, reliability, usability, security, and maintainability are treated as project requirements and summarized in Table 2.2.

ID	Quality	Target indicator	Validation
NFR-01	Performance	Keep the loop near real time and limit alert delay.	Runtime observation under representative feed scenarios.
NFR-02	Scalability	Support independent control for multiple feed contexts.	Feed registry behavior and state-transition checks.
NFR-03	Reliability	Recover gracefully from camera drops and webhook failures.	Reconnect paths and retry/fallback test scenarios.
NFR-04	Usability	Keep setup and monitoring clear for non-developer users.	Dashboard integration tests and workflow walkthroughs.
NFR-05	Security	Restrict integration access through secrets and credentials.	Webhook secret validation and credential checks.
NFR-06	Maintainability	Keep the architecture modular and testable.	Design review and test-suite coverage by module.

Table 2.2: Non-functional requirement quality matrix

2.3 Product Backlog and Sprint Organization

The product backlog was prioritized according to business value and technical dependency, then organized into four sprints. The first sprint focused on perception and queue analytics because the dashboard and alerts depend on stable metrics. The second sprint added source management and live services. The third sprint turned those services into

a usable web dashboard. The final sprint connected alerts to n8n and Telegram, then evaluated the complete system.

2.4 Global Use Case Diagram

The global use case diagram gives a high-level view of how the administrator and manager interact with the platform during configuration, monitoring, and alert handling. It is shown in Figure 2.1.

Use Case Diagram: Chapter 2

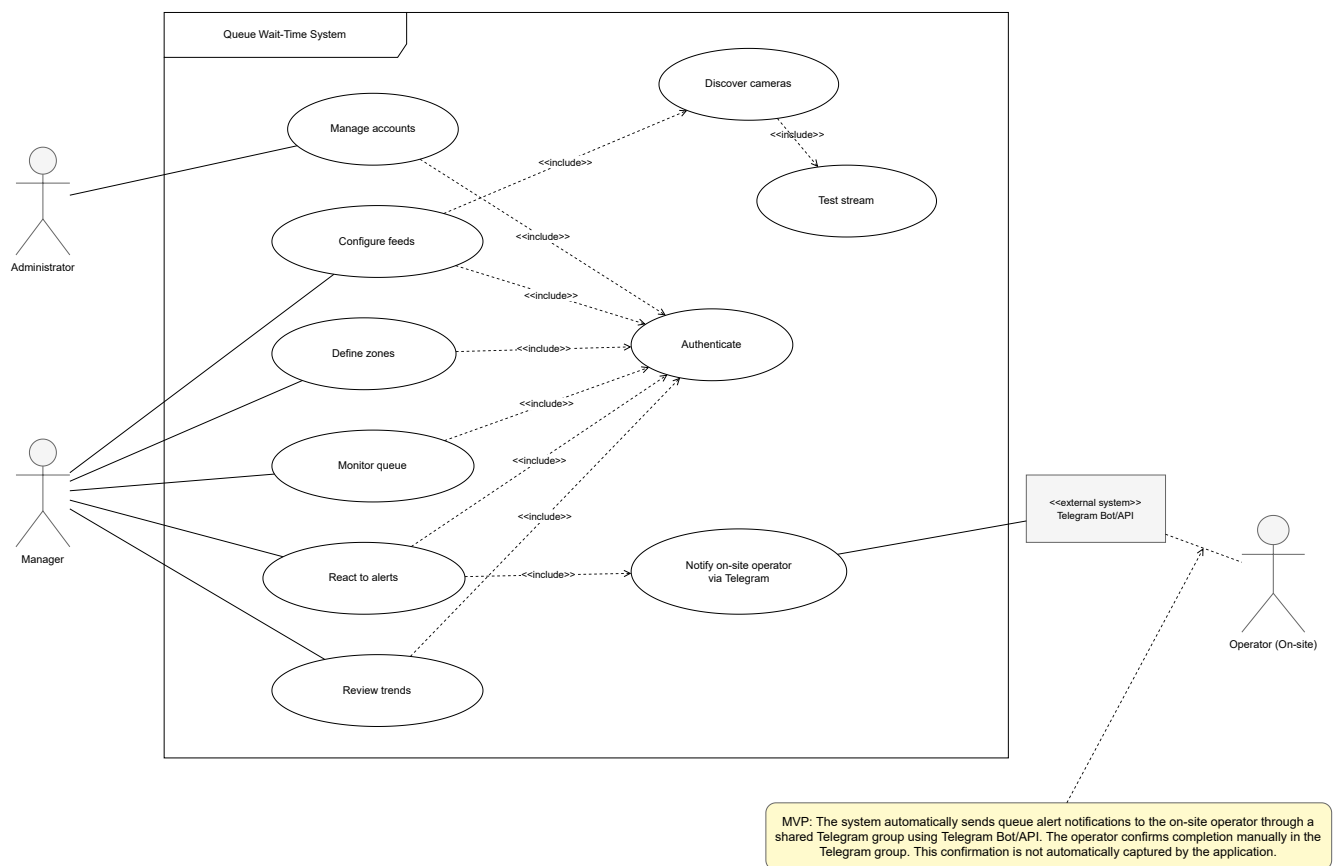


Figure 2.1: Global use case diagram of the queue wait-time estimation system.

At the system level, the use cases can be grouped into four interactions: configuration, live monitoring, analytics interpretation, and alert handling/resolution, with the main actor responsible for each use case listed in Table 2.3.

Use case ID	Use case summary	Primary actor
UC-01	Create, update, or remove manager accounts.	Administrator
UC-02	Register and configure a video feed source.	Manager
UC-03	Define or update queue zone geometry.	Manager
UC-04	Monitor live queue metrics and feed status.	Manager
UC-05	Receive and interpret warning or critical alerts.	Manager
UC-06	Automatically notify the on-site operator via Telegram when an alert is detected.	System automatic behavior
UC-07	Review trends and performance indicators.	Manager

Table 2.3: Primary global use cases

2.5 Project Environment

2.5.1 Hardware and Software Environment

The hardware environment used during development is shown in Table 2.4.

Component	Type
Processor	Intel Core i5 12th Generation
Memory (RAM)	16 GB
Graphics card	NVIDIA GeForce RTX 4060
Storage	512 GB SSD

Table 2.4: Hardware environment

2.5.2 Development Tools

The project also relied on a small set of tools for editing, versioning, diagramming, container support, and report generation. Keeping these tools separate from the technology

stack makes it easier to distinguish the working environment from the components used by the final system, as listed in Table 2.5.

Tool	Role in the project	Typical use
VS Code [8]	Main editor for the backend, frontend, and report sources.	Day-to-day implementation, review, and navigation.
Git / GitHub [9], [10]	Version control and change history tracking.	Commit management, branching, and collaboration.
draw.io [11]	Diagram authoring for architecture and UML models.	Use case, UML class, and system architecture diagrams.
Docker / Docker Compose [12]	Local service orchestration and reproducible setup.	Supporting services such as MediaMTX and n8n.

Table 2.5: Development tools used in the project

2.5.3 Technologies Used

The technology stack follows the main stages of the system, as summarized in Table 2.6.

Layer	Main technologies	Role in the project
Vision pipeline	YOLO26, ByteTrack, and OpenCV [2], [13], [14]	Detects people, tracks them across frames, and prepares the visual input for queue analysis.
Queue analytics	NumPy, Pandas, SciPy, and M/M/1 modeling [1], [15], [16], [17]	Computes arrival rate, service rate, and wait time.
Backend API	FastAPI and Pydantic v2 [18], [19]	Exposes typed endpoints, validates payloads, and coordinates the backend services.
Live transport	WebRTC for IP cameras, MediaMTX WHEP gateway, and MJPEG playback for local MP4 videos [20], [21]	Delivers low-latency video streaming for camera feeds while keeping a simpler HTTP playback path for uploaded videos.
Frontend	React, TypeScript, Vite, and Tailwind CSS [22], [23], [24], [25]	Implements the dashboard, user interactions, and responsive presentation layer.
Automation	n8n workflows and Telegram notifications [26], [27]	Routes alerts and sends operational messages to users.
Persistence	SQLite [28]	Stores metrics, metadata, and alert history for later analysis.

Table 2.6: Technologies used in the project

This selection supports the complete path from perception through automation while keeping each stage understandable on its own.

2.6 Software Architecture

2.6.1 Overall Architecture

The architecture is described as four stages that match the flow of data through the system, from video acquisition to alert delivery. Figure 2.2 illustrates this flow.

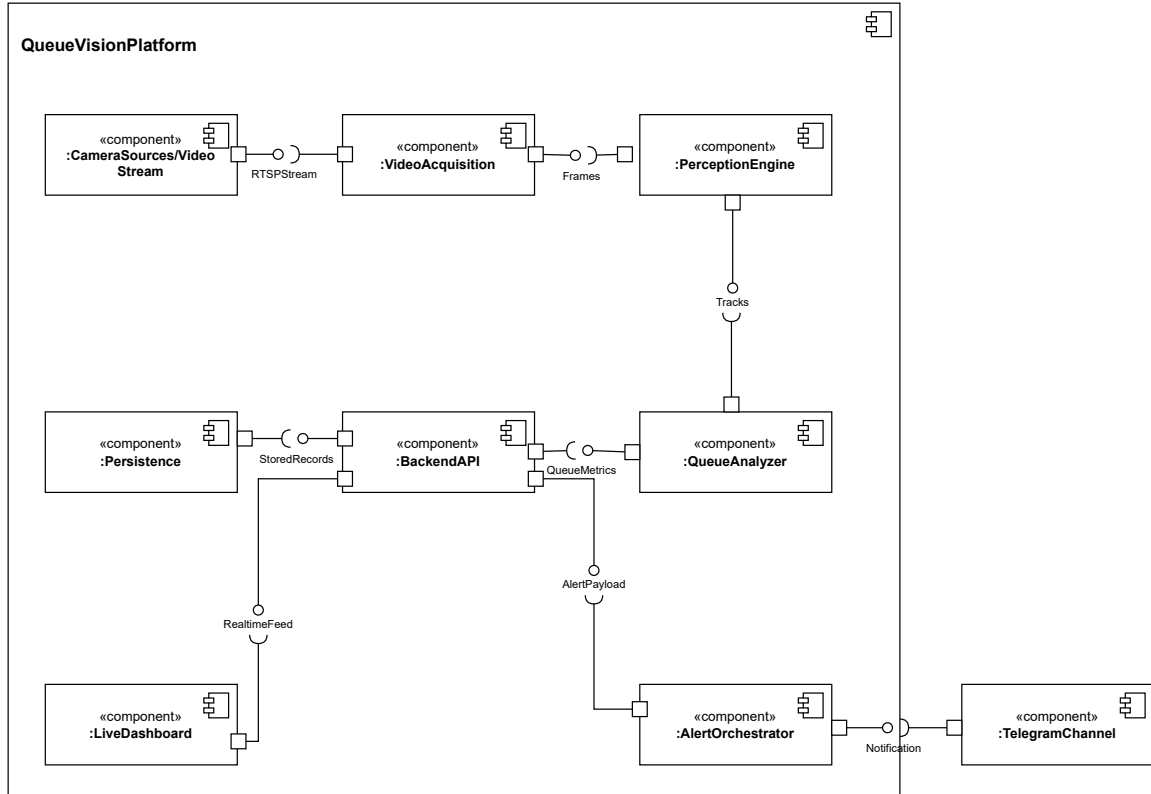


Figure 2.2: Overall system architecture from video acquisition to alert delivery.

1. **Video acquisition:** camera sources and video streams (RTSP, ONVIF, and local files) feed into the video acquisition component, which handles frame capture and stream management [29], [30].
2. **Perception:** the perception engine runs detection and tracking on each captured frame and exposes tracked identities through a defined interface to the queue analyzer.
3. **Analysis:** the queue analyzer receives tracked events and computes queue metrics (λ , μ , W) using adaptive wait-time estimation logic.
4. **Output and integration:** the backend API acts as the hub that receives queue metrics and distributes them across three outputs: the live dashboard via a real-time feed, the persistence layer (SQLite) for historical records, and the alert orchestrator (n8n) which routes alert payloads to the Telegram notification channel.

This separation keeps the four stages easier to validate. Queue formulas depend

on perception events, the API depends on the metric contract, and the dashboard, persistence, and alert orchestration consume the same project-level outputs without being tightly coupled to one another.

2.6.2 Backend Architecture

The backend exposes REST and WebSocket interfaces for feed lifecycle control and live metrics. It coordinates the main project responsibilities: source management, queue analysis, threshold evaluation, alert delivery, and live dashboard updates.

Feed-level orchestration lets each camera be started, stopped, or refreshed independently. If one feed fails, the others keep running, and the dashboard still receives a predictable event stream.

Persistence and integration are treated as core responsibilities rather than add-ons. Stored state preserves operational context, while webhook delivery turns queue events into external notifications.

2.6.3 Frontend Architecture

The frontend is built as a component-based interface with feed-oriented dashboard composition and WebSocket-driven synchronization. Its role is to make setup, monitoring, and analytics usable for managers without exposing the complexity of the underlying services.

This separation makes the dashboard easier to extend when new feed controls or indicators are added, while preserving a stable user experience for monitoring and setup.

2.7 Conclusion

This chapter established the functional baseline, the quality constraints, and the architecture choices that guide the rest of the report. The next chapters show how these requirements were implemented through perception, connectivity, dashboard monitoring, automation, and evaluation work.

Sprint 1: Perception and Queue Analytics

Table of Content

3.1	Introduction	26
3.2	Sprint Objectives and Backlog	26
3.3	Sprint Backlog	26
3.4	Design	28
3.4.1	Perception and Analytics Flow	28
3.5	Implementation	31
3.5.1	Perception Pipeline	31
3.5.2	Queue Event Windows and Rate Estimation	31
3.5.3	Wait-Time Logic and Adaptive Estimation	32
3.5.4	Runtime Metric Contract	32
3.6	Conclusion	33

3.1 Introduction

The first sprint answered the most basic question of the project: can a video feed be transformed into queue indicators that are stable enough for later use? The work therefore started with people detection, frame-to-frame tracking, and zone filtering before moving to arrival rate, service rate, and wait-time estimation.

Delivering perception and analytics together was important because the later dashboard and alerting work needed more than raw counts. They needed a consistent metric output that already represented the queue zone, the tracked identities, and the estimated flow of people.

3.2 Sprint Objectives and Backlog

Sprint 1 had two linked objectives. The first was to obtain a reliable perception baseline by detecting people, preserving identity across frames, and ignoring movement outside the queue zone. The second was to convert those observations into operational metrics that a manager could interpret later in the dashboard. The sprint backlog separates these manager-facing outcomes from the technical tasks needed to support them.

3.3 Sprint Backlog

The Sprint 1 backlog separates manager-facing user stories from technical tasks. User stories represent stakeholder needs, while technical tasks describe the internal work required to make those needs measurable. The user stories in Sprint 1 represent manager needs for reliable perception and analytics, as shown in Table 3.1.

SPRINT 1: PERCEPTION AND QUEUE ANALYTICS

User Story ID	User Story	Task ID	Task
US1.1	As a manager, I want to detect people in the video stream so that queue observations start from a reliable signal.	T1.1	Configure the detector to identify people in live and recorded video.
US1.2	As a manager, I want to keep the same person identifiable across frames so that counts remain consistent.	T1.2	Integrate tracking to preserve identity across consecutive frames.
US1.3	As a manager, I want to restrict counting to queue zones so that irrelevant movement does not affect the queue state.	T1.3	Define queue zones as polygons and filter detections by zone.
US1.6	As a manager, I want queue rates estimated from tracked people so that the dashboard can interpret flow intensity.	T1.6	Group tracked events into short windows and compute arrival and service rates.
US1.7	As a manager, I want wait time computed adaptively so that the result remains useful in real conditions.	T1.7	Apply the M/M/1 formula when arrivals are below service capacity and fallback to occupancy-based estimation otherwise.
US1.9	As a manager, I want a clear metric output so that the dashboard can reuse these values without reprocessing them.	T1.9	Publish rate estimates, wait time, and people-in-zone occupancy.

Table 3.1: Sprint 1 user stories

Supporting technical tasks address internal implementation requirements that enable these user stories, as detailed in Table 3.2.

Task ID	Task	Subtask ID	Subtask
T1.4	Connect capture, detection, tracking, zone filtering, and display into one continuous flow.	ST1.4.1	Implement runtime orchestration that integrates all perception components.
T1.5	Expose runtime settings so that the pipeline can be adapted without code changes.	ST1.5.1	Provide configurable parameters for frame handling and visualization.
T1.8	Add guards for empty windows, low counts, and bounded fallback output to protect calculations from division-by-zero and noise.	ST1.8.1	Implement defensive bounds checking in the analytics layer.

Table 3.2: Sprint 1 technical tasks

3.4 Design

3.4.1 Perception and Analytics Flow

The Sprint 1 workflow can be read at two levels: the overall path from frame ingestion to metric output, and the detailed transformation from tracked identities to queue rates. This distinction helped keep the visual processing concerns separate from the waiting-time calculation.

At the perception level, the system handles three key scenarios: when a person enters the queue zone, they are treated as a possible arrival; when the same person appears in consecutive frames, the system maintains stable identification to avoid double-counting; and when people appear outside the zone, they are filtered out to keep measurements focused on the queue area.

At the queue-operation level, the system handles three practical situations. When arrivals remain below service capacity, the M/M/1 formula provides a rate-based estimate. When arrivals approach or exceed service capacity, the estimate switches to a

conservative occupancy-based value. When no service events are available, the output remains bounded instead of producing unstable values. This adaptive behavior keeps a single wait-time indicator usable across changing queue conditions.

At the high level, Figure 3.1 shows the end-to-end flow from a video source (local file or live camera stream) through detection, tracking, zone filtering, and finally to queue analysis that computes arrival rate, service rate, and expected wait time.

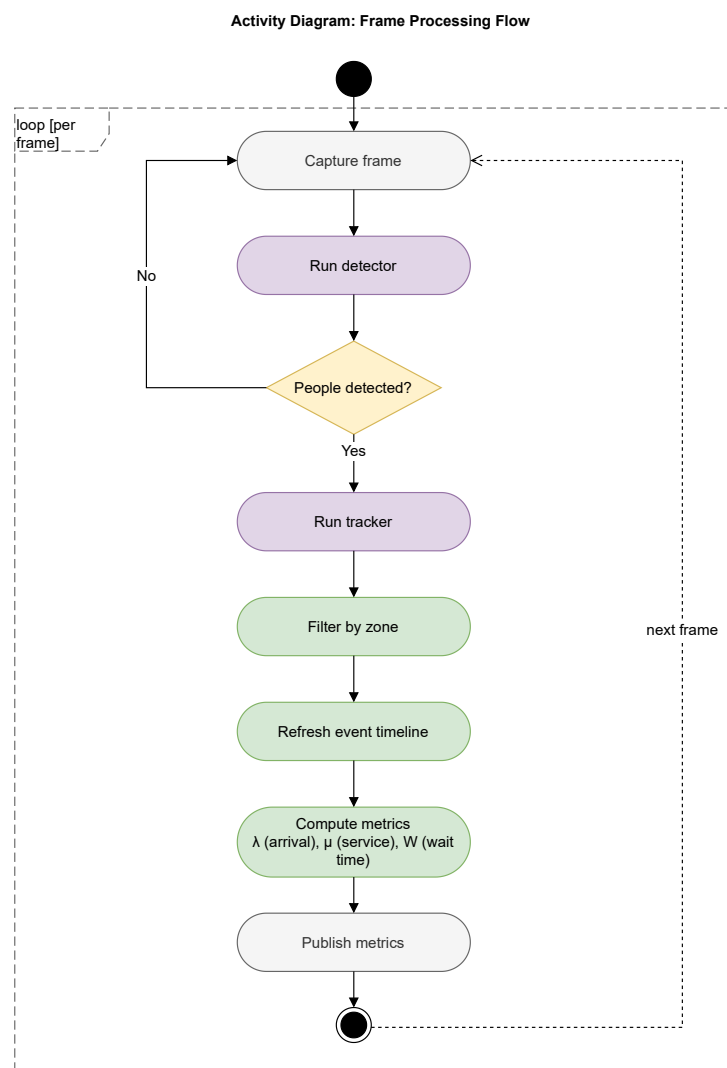


Figure 3.1: Sprint 1 activity flow from frame ingestion to queue metrics output.

At the detailed level, Figure 3.2 focuses on the queue analytics transformation that converts tracked people inside the zone into the metric payload consumed by the dashboard and later automation layers. The analytics step compares current tracked identifiers with the previous observation window to identify arrivals and departures.

These events are stored in short windows, then converted into rates (λ and μ). Before applying the wait-time formula, the system checks whether enough events were observed, whether μ is positive, and whether arrivals remain below service capacity. If these guards are satisfied, it uses the M/M/1 expression; otherwise, it falls back to a bounded occupancy-based estimate.

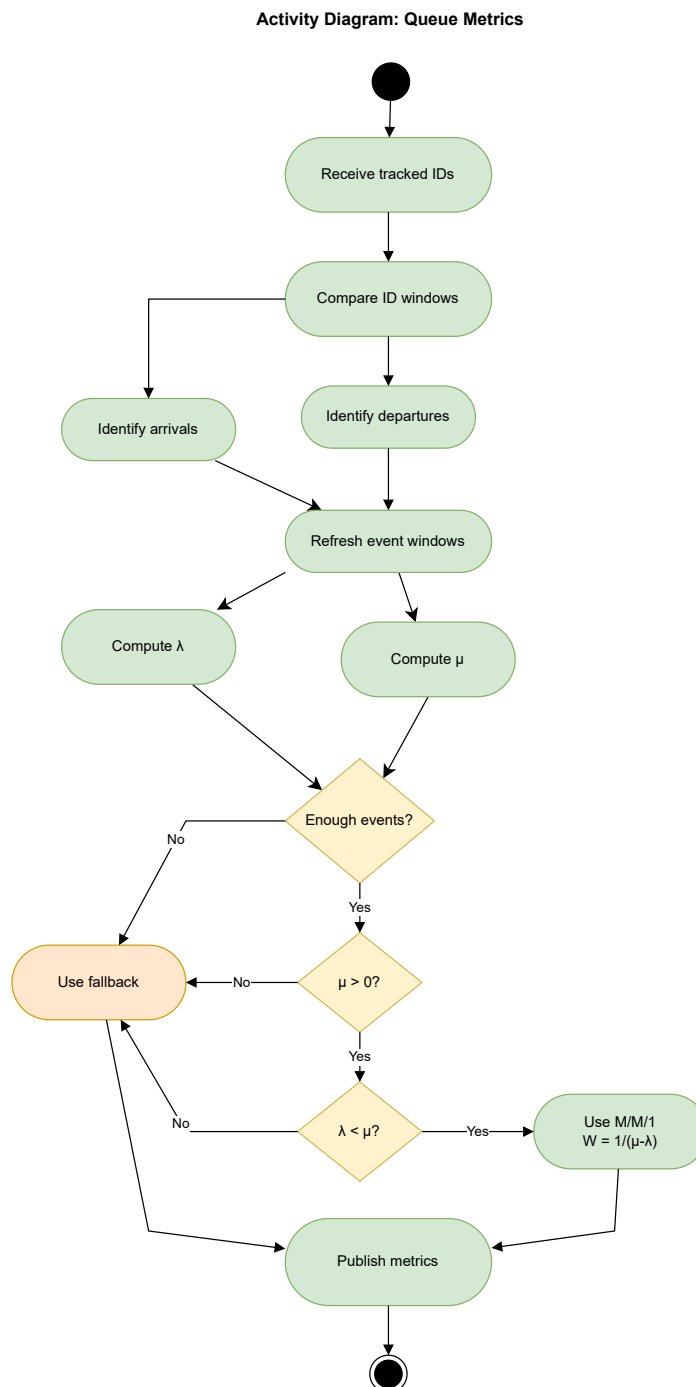


Figure 3.2: Sprint 1 analytics activity diagram for queue metric generation.

3.5 Implementation

3.5.1 Perception Pipeline

The perception pipeline combines detection, tracking, and zone filtering. YOLO identifies people in each frame, while ByteTrack preserves identity across consecutive frames. This identity continuity is necessary for arrival and departure counting, because a person should not be counted again simply because they appear in the next frame. Zone-aware filtering then restricts the measurement to the polygon-defined queue area, so unrelated movement elsewhere in the scene does not affect the queue metric.

The same processing path supports local files, webcams, and live camera streams. It can adjust frame-processing frequency, display overlays for detections, tracker identities, queue zones, and computed metrics, or run without visual output when the results are needed by later system components.

3.5.2 Queue Event Windows and Rate Estimation

The queue-analysis step uses short time windows to count arrivals and departures. Let N_a and N_d be the number of events observed in windows T_a and T_d :

$$\lambda = \frac{N_a}{T_a}, \quad \mu = \frac{N_d}{T_d}$$

From these counts, the system estimates the arrival rate and the service rate. To avoid unstable values at the beginning of a run, the implementation waits until enough events have been observed before returning non-zero rates.

3.5.3 Wait-Time Logic and Adaptive Estimation

The system estimates wait time using an adaptive formula that responds to queue conditions:

$$W = \begin{cases} \frac{1}{\mu - \lambda} & \text{if } \lambda < \mu \text{ and } \mu > 0 \\ \frac{L}{\mu} & \text{otherwise} \end{cases}$$

where L is the current number of people in the zone. We use the M/M/1 sojourn-time expression because the project models each monitored lane as a single service channel with one effective queue, which matches the cashier-style setting studied in the report. Other models were less appropriate here: M/M/c would assume multiple parallel servers, which is not the default lane setup, and Little's Law is useful for validation but does not directly produce a forward wait-time estimate from the instantaneous rates alone.

The adaptive formula keeps the output interpretable. Under normal load, the estimate comes from the rate-based expression. Under overload, it falls back to a conservative value based on the current queue size. When no service events are observed, the result is bounded instead of allowing division by zero or an unrealistic wait-time value.

3.5.4 Runtime Metric Contract

The final metric output includes:

- point estimates (λ, μ, W) ,
- people-in-zone occupancy.

These values are then reused by logs, overlays, webhook payloads, and dashboard events [1], [14].

3.6 Conclusion

Sprint 1 delivered the first complete path from raw video to queue indicators. Detection, tracking, and zone filtering produced a usable perception baseline, while the queue analytics step converted tracked observations into arrival rates, service rates, and adaptive wait-time estimates.

The main result of the sprint was not only that people could be counted, but that the count was translated into a reusable queue signal. The adaptive wait-time logic made this signal more robust: it used the rate-based model when the queue was stable, moved to an occupancy-based estimate when the queue became overloaded, and stayed bounded when service events were missing.

By connecting perception to interpretation in one sprint, the project avoided building the dashboard on top of raw visual counts. The metric output established here became the foundation for web monitoring, persistence, and automated alerting in the following sprints.

Table of Content

4.1	Introduction	35
4.2	Sprint Objectives and Backlog	35
4.3	Design	36
4.3.1	Backend Domain Model and Class Structure	36
4.4	Implementation	37
4.4.1	Source Integration and Camera Discovery	37
4.4.2	API and Live Transport	38
4.4.3	Persistence, State Recovery, and System Resilience	41
4.5	Conclusion	41

4.1 Introduction

Sprint 2 moved the project from a local video-processing pipeline to a service that could manage several feed sources. The main challenge was no longer only computing queue metrics, but making camera onboarding, feed control, live events, and video delivery available to the rest of the system in a consistent way.

This sprint also prepared the ground for the web dashboard. Managers would later need to discover cameras, configure sources, start or stop feeds, and keep watching a stream even when the preferred transport was unavailable.

4.2 Sprint Objectives and Backlog

The sprint objectives focused on four manager needs: secure connection to IP cameras, automatic camera discovery, centralized feed management, and reliable live delivery. These needs are captured in Table 4.1.

User Story ID	User Story	Task ID	Task
US2.1	As a manager, I want IP camera streams connected securely so that live sources can be reused reliably.	T2.1	Add authentication, transport control, reconnection, and snapshots for RTSP feeds.
US2.2	As a manager, I want cameras discovered automatically so that stream addresses do not need to be hardcoded.	T2.2	Detect IP cameras and resolve their stream addresses.
US2.3	As a manager, I want feed management exposed through the API so that camera lifecycle actions are centralized.	T2.3	Expose feed control, metadata handling, and diagnostic operations.
US2.4	As a manager, I want live delivery to stay available so that the dashboard can keep showing the active feed.	T2.4	Support live delivery through real-time events and a layered video transport strategy.

Table 4.1: Sprint 2 user stories

4.3 Design

4.3.1 Backend Domain Model and Class Structure

The backend model was organized around the entities that appear in a real monitoring scenario: establishments, managers, feed configurations, video sessions, queue metrics, and alerts. Establishments group both managers and feeds, while managers operate only on the feeds assigned to their scope. This organization keeps access control and feed recovery consistent when the system restarts.

The core entities and their relationships are illustrated in Figure 4.1.

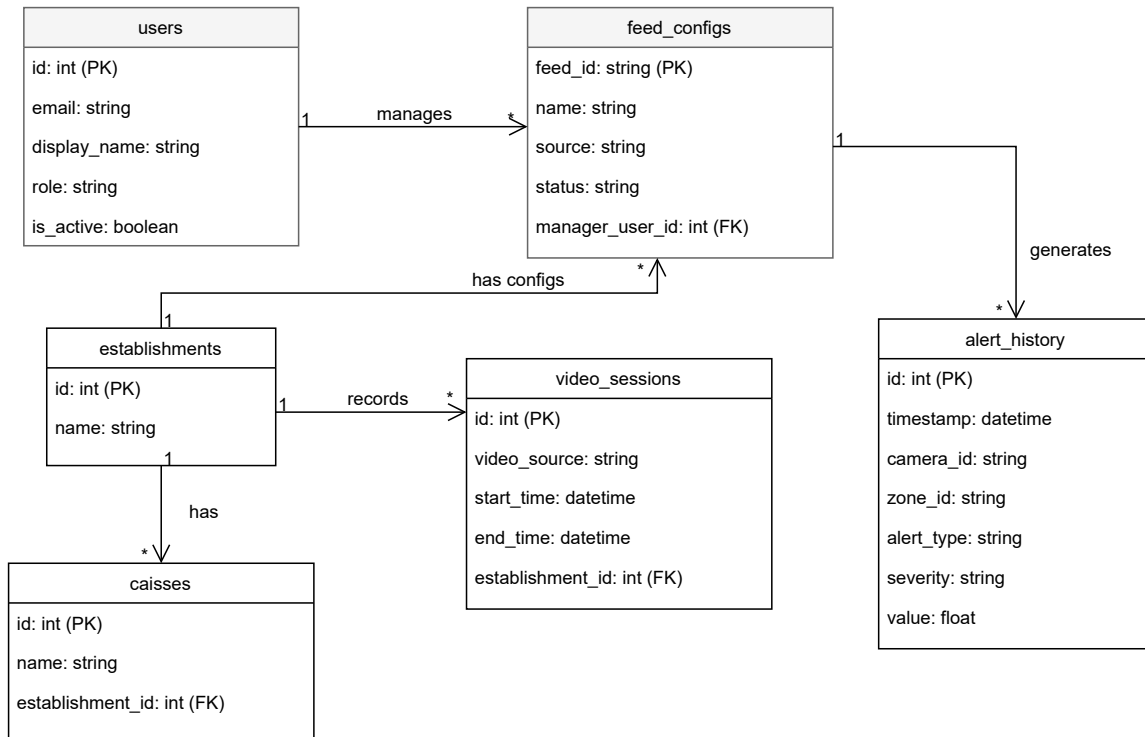


Figure 4.1: Sprint 2 backend class diagram showing core entities for feed orchestration and persistence.

4.4 Implementation

4.4.1 Source Integration and Camera Discovery

The system supports several source types, but IP cameras required the most careful handling. RTSP streams need credentials, transport selection, connection testing, and reconnection behavior. Local files and webcams were kept on simpler paths so that camera-specific constraints would not complicate every source.

Camera discovery uses ONVIF to locate devices and resolve stream information without manually entering hardcoded addresses. The workflow starts with WS-Discovery over multicast UDP, then resolves media services and RTSP stream URIs. It also handles different ONVIF camera behaviors: some cameras expose media services directly, while others require an intermediate capability lookup before the stream address can be requested. The workflow was validated against a Digital Watchdog IP camera on a shared network, which confirmed that discovery, stream resolution, and validation

could work in a realistic setup.

The discovery process gives the manager practical actions: find available cameras, test a connection, resolve the stream, and capture a snapshot before defining the queue zone. Credentials are handled without exposing them in logs, and the transport choice can adapt to the network conditions and source type.

The logical workflow of this discovery process is illustrated in Figure 4.2.

The protocol-level details are captured in Figure 4.3, which illustrates the three-phase ONVIF sequence: multicast UDP discovery, direct media service access, and device service indirection.

4.4.2 API and Live Transport

The backend exposes REST and WebSocket interfaces for feed management, source onboarding, metadata operations, and live event delivery. Access rules are applied at the session level: managers interact only with their assigned feeds, while administrators keep a broader governance role. This prevents the dashboard from showing controls that the current user should not access.

Live transport is selected according to the source type. IP camera feeds prioritize WebRTC because the monitoring screen benefits from lower latency. Local MP4 videos use MJPEG playback, which is simpler and sufficient for file-based demonstrations. When a preferred transport is unavailable, the backend reports the condition explicitly so the dashboard can show a fallback path instead of waiting silently.

Runtime control coordinates feed state, saved configuration, worker lifecycle, and live event broadcasting. The dashboard receives feed status changes, metric updates, alerts, and warnings through the live channel. For local MP4 videos, a shared MJPEG stream preserves viewing continuity during runtime mode transitions.

Activity Diagram: Camera Discovery

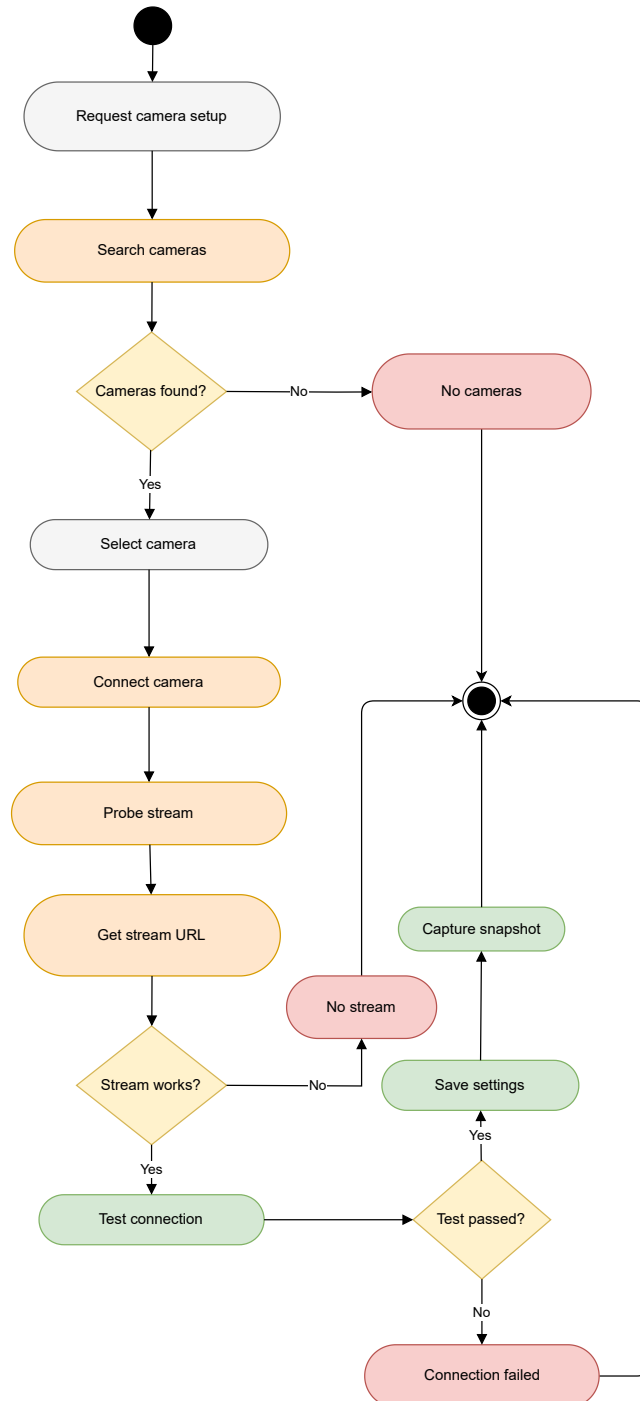


Figure 4.2: Camera discovery activity diagram: ONVIF discovery, RTSP resolution, and stream validation.

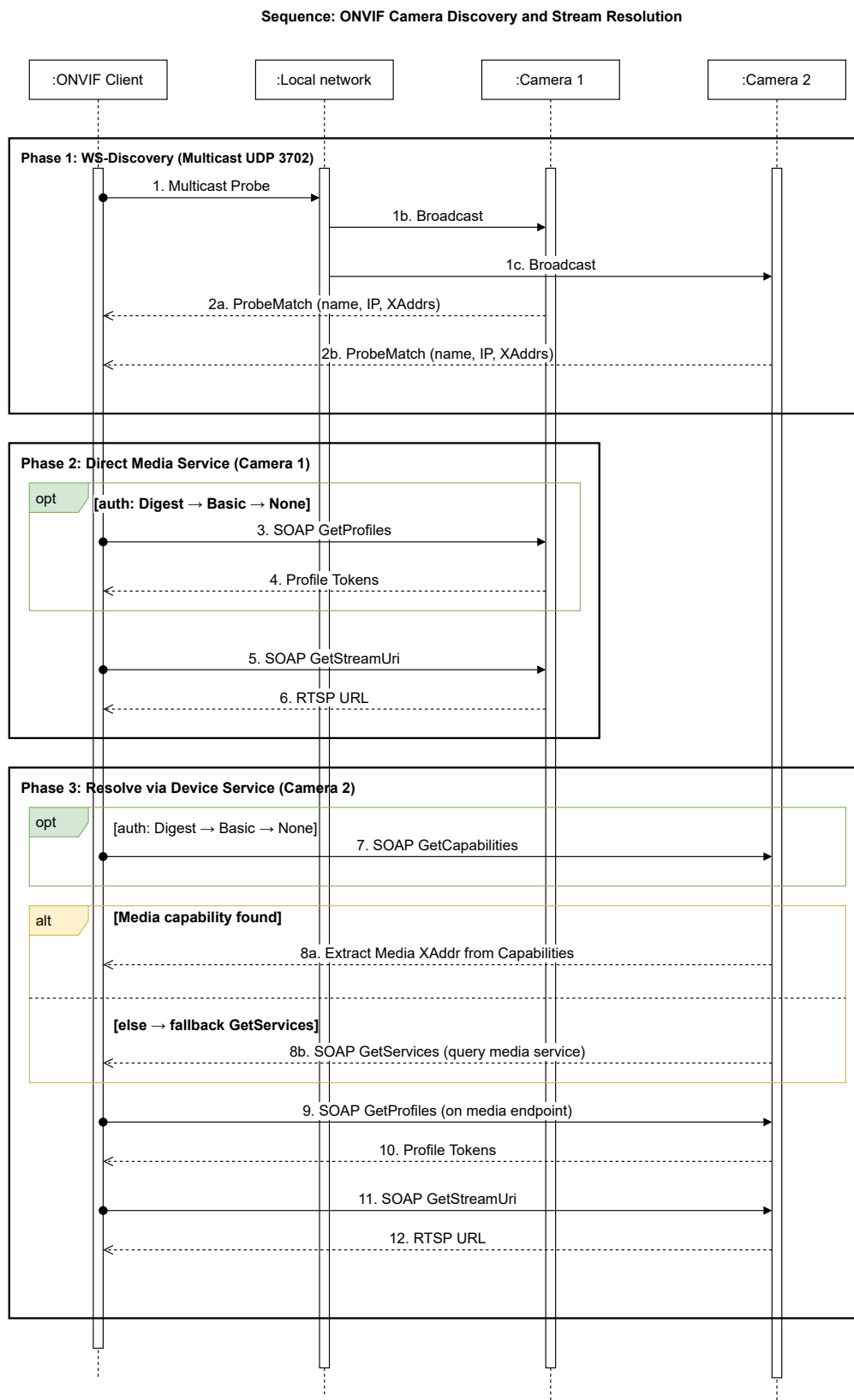


Figure 4.3: ONVIF protocol sequence: WS-Discovery broadcast, direct media service resolution, and device service indirection.

4.4.3 Persistence, State Recovery, and System Resilience

SQLite stores establishments, managers, feed configurations, video sessions, alert history, and system state. The structure keeps feed definitions attached to the correct establishment and manager, which is important when several feeds or users exist in the same deployment.

On startup, saved feed definitions are reloaded and unfinished sessions are reset to a consistent state. This avoids orphaned sessions and lets the application recover its last known configuration after a restart. In practice, Sprint 2 showed that camera connectivity is not only about reaching an RTSP address; the system also needs to remember feed state, report transport failures clearly, and recover from interruptions without losing the manager's configuration.

4.5 Conclusion

Sprint 2 turned the perception prototype into a managed service layer for camera discovery, feed lifecycle control, live events, and video delivery. The work addressed three project-level challenges: connecting heterogeneous sources through ONVIF and RTSP, keeping access and feed state consistent, and providing fallback-aware live delivery for the dashboard.

The sprint established the runtime foundation used by the web interface in Sprint 3. By the end of the sprint, the project could move from a configured source to live queue monitoring data without requiring the manager to handle camera addresses or transport details manually.

Table of Content

5.1 Introduction 44

5.2 Sprint Objectives and Backlog 44

5.2.1 User Stories 44

5.2.2 Technical Tasks 46

5.3 Design 46

5.3.1 Visual Design and Interface Architecture 46

5.3.2 Dashboard Data Flow and Integration Architecture 47

5.3.3 Design Rationale 47

5.4 Implementation 47

5.4.1 Project Setup and Layout Skeleton 47

5.4.2 Styling and Reusable Interface Rules 49

5.4.3 API Integration 49

5.4.4 Authentication and Role-Based Navigation 49

5.4.5 Dashboard State Orchestration and Live Update Reliability 50

5.4.6 Live Video Delivery and Transport Selection 52

5.4.7 Dashboard UI Integration 52

5.4.8 Manager Onboarding Workflow 52

5.4.9 Analytics and Historical Reporting 53

5.4.10 Integration Testing 55

5.5	Conclusion	56
-----	----------------------	----

5.1 Introduction

Sprint 3 turned the backend capabilities from Sprint 2 into a manager-facing web dashboard. The work began with the visual structure of the application, then moved to authentication, feed onboarding, live metrics, transport selection, and fallback behavior.

The main objective was to make monitoring usable in one place. A manager should be able to sign in, prepare a feed, define the queue zone, launch monitoring, follow live indicators, and review analytics without switching between separate tools.

5.2 Sprint Objectives and Backlog

Sprint 3 combined interface design with live system integration. The design work defined the visual language, page structure, and reusable interaction patterns. The integration work connected those screens to authentication, feed management, WebSocket updates, video transport, and fallback polling. The sprint backlog organized these goals into manager-facing and administrator-facing user stories.

5.2.1 User Stories

Table 5.1 lists the manager and admin requirements for Sprint 3.

SPRINT 3: WEB DASHBOARD AND LIVE MONITORING

User Story ID	User Story	Task ID	Task
US3.1	As a manager, I want a consistent visual style so that the dashboard feels like one coherent product.	T3.1	Define colors, typography, spacing, and reusable component rules.
US3.2	As a manager, I want the main page layouts prepared so that onboarding, monitoring, and analytics screens are available.	T3.2	Prepare the landing page, dashboard, analytics, and settings layouts.
US3.3	As an admin, I want to create and maintain manager accounts so that system access is centrally controlled.	T3.3	Build the admin dashboard for manager account creation, editing, and deletion.
US3.4	As a manager, I want the dashboard connected to backend responses so that the interface reflects live operational data.	T3.4	Implement the API integration layer and typed request handling.
US3.5	As a manager, I want to manage feeds from the interface so that feed lifecycle actions stay centralized.	T3.5	Connect login state, protected routes, and feed control actions to the dashboard.
US3.6	As a manager, I want live updates without refreshing so that changes are visible immediately.	T3.6	Add websocket-driven live state updates and dashboard orchestration.
US3.7	As a manager, I want a suitable video transport chosen automatically so that the feed remains usable when conditions change.	T3.7	Add transport selection, fallback streaming, and reconnection logic.
US3.8	As a manager, I want the dashboard to stay usable during interruptions so that monitoring does not stop during transient failures.	T3.8	Add fallback polling and stale-data guards.

Table 5.1: Sprint 3 user stories

5.2.2 Technical Tasks

Table 5.2 lists the internal implementation work for integration and testing.

Task ID	Task	Subtask ID	Subtask
T3.10	Set up the frontend project with shared components and page scaffolding so that the interface can grow without duplication.	ST3.10.1	Create reusable UI component library and establish project structure.
T3.11	Validate dashboard setup and onboarding flows with integration tests to ensure the integration behaves correctly end to end.	ST3.11.1	Implement scenario-based tests for dashboard lifecycle and error paths.

Table 5.2: Sprint 3 technical tasks

5.3 Design

5.3.1 Visual Design and Interface Architecture

The interface follows a “Terminal Minimal” style suited to monitoring work. Dark surfaces reduce visual fatigue during long sessions, while semantic colors distinguish normal, warning, and critical states. Cards and panels are used to group feed summaries, KPI values, recent activity, and configuration steps without changing the manager’s mental model from page to page.

The application is organized around five main pages: landing, dashboard, analytics, settings, and authentication. Defining this navigation early helped separate design decisions from live-data behavior. Once the main screens were stable, integration work could focus on whether the dashboard reacted correctly to feed changes, alerts, and transport failures.

5.3.2 Dashboard Data Flow and Integration Architecture

The dashboard data flow starts after authentication. The interface retrieves feed configuration and system health, then opens a persistent WebSocket connection for live updates. The backend sends an initial metrics snapshot, followed by feed lifecycle events, metric changes, alerts, and warnings. Feed actions such as launch, stop, or update are sent as REST commands, then reflected back through the live event stream.

This flow gives the dashboard two ways to remain informative. During normal operation, WebSocket events keep the metrics current without a page refresh. If live events stop arriving, the dashboard switches to periodic polling so the manager can still see feed state and recent metrics. For video playback, WebRTC is used for IP camera feeds when available, while MJPEG remains available for local videos and fallback views.

Figure 5.1 illustrates this complete data flow during setup, runtime operation, and transport negotiation.

5.3.3 Design Rationale

The design-first approach reduced confusion during integration. Because navigation, page responsibilities, and repeated interface elements were already defined, later work could focus on behavior: whether feeds launched correctly, whether metrics updated at the right time, and whether fallback states were understandable to the manager.

5.4 Implementation

5.4.1 Project Setup and Layout Skeleton

The design specification was translated into an application skeleton that includes protected navigation, the main dashboard layout, and shared interface elements across

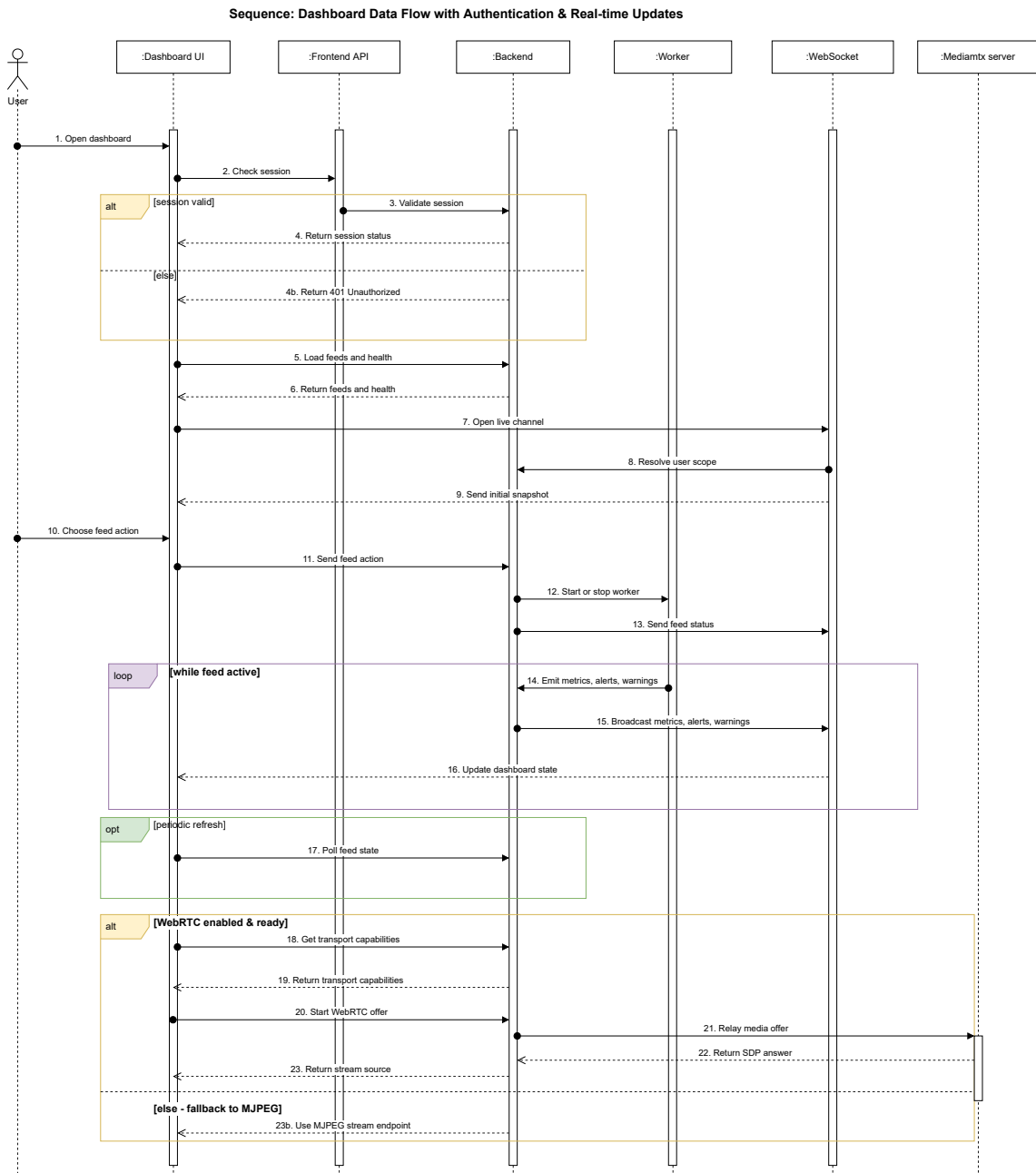


Figure 5.1: Dashboard data flow sequence showing protected access (alt), metric broad-casts (loop), fallback polling (opt), and video transport negotiation (alt).

pages.

The page order follows the operational workflow: onboarding comes before configuration, configuration comes before launch, and launch leads into supervision. This makes the dashboard feel like one continuous monitoring path rather than a collection of unrelated screens.

5.4.2 Styling and Reusable Interface Rules

The visual system was defined once and reused across the dashboard. Spacing, typography, status colors, surfaces, and panel hierarchy follow the same rules in onboarding, monitoring, analytics, and administration screens. This helped avoid design drift as more live behavior was added.

5.4.3 API Integration

The dashboard communicates with the backend through a dedicated integration boundary. Feed lists, dashboard summaries, authentication state, and management actions are handled as explicit responses rather than informal page-specific data. This keeps the user interface focused on presentation and interaction while the communication layer handles request state and errors.

5.4.4 Authentication and Role-Based Navigation

Sprint 3 integrated authentication and role-based access into the dashboard. Session changes clear cached dashboard data, protected routes redirect users according to role, administrators reach the manager administration interface, and managers reach the monitoring dashboard. The purpose is simple from the user's perspective: each role sees the screens that match its responsibilities.

Three key screens support this flow: the manager sign-in screen, the manager account creation screen, and the administrator account-governance interface. These

screens are shown in Figures 5.2, 5.3, and 5.4. Together, they keep authentication, account creation, and role-based routing within the same product experience.

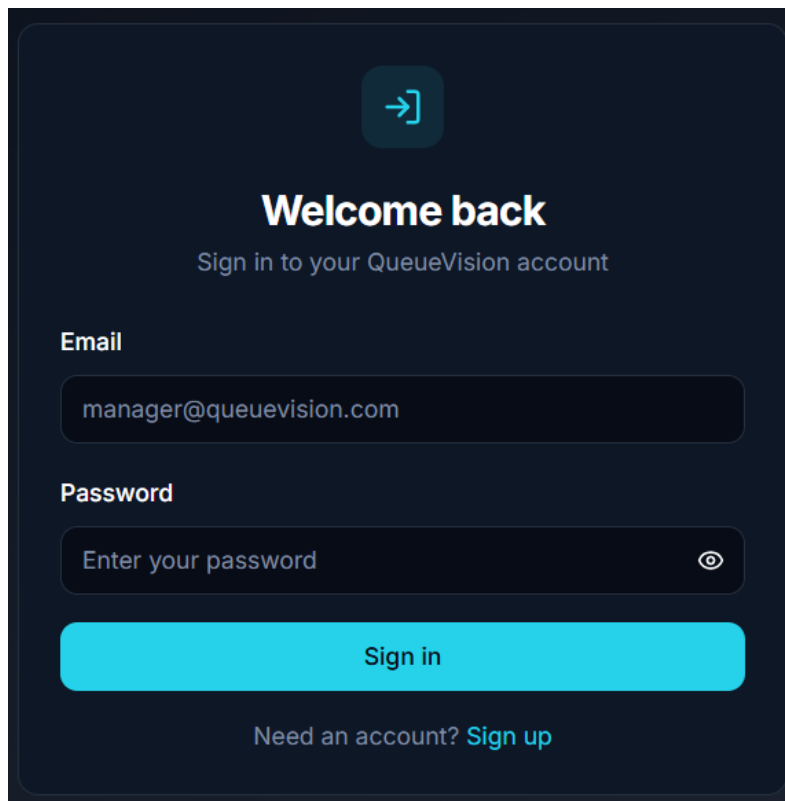


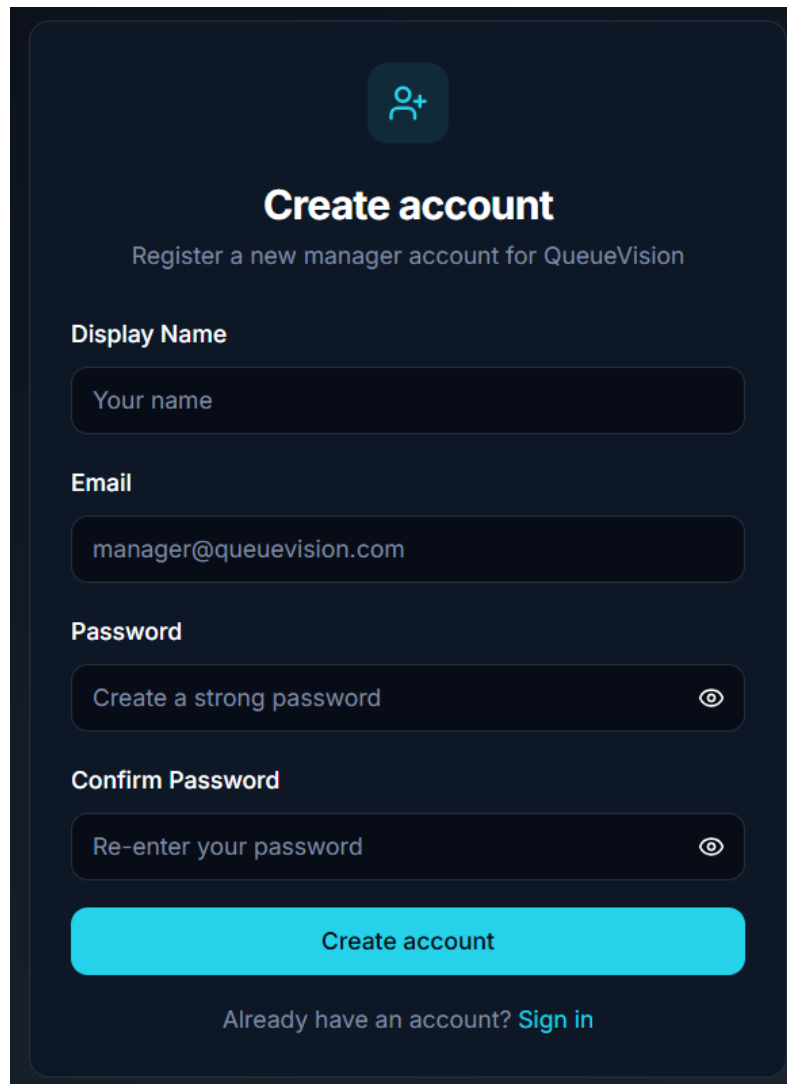
Figure 5.2: Manager sign-in screen for dashboard access.

5.4.5 Dashboard State Orchestration and Live Update Reliability

The dashboard combines feed data into operational indicators such as online feed count, total people, average wait time, and alert count. These indicators are recalculated as feed events arrive, so the monitoring screen reflects the current state of the system rather than a static page load.

Live reliability relies on several behaviors:

- reconnection delays to avoid repeated rapid retries during outages,
- heartbeat monitoring to detect connection loss,
- stale-data guards so old metrics are not presented as fresh,
- fallback polling when live events become unavailable,



The image shows a 'Create account' form for QueueVision. At the top is a user icon. The title 'Create account' is centered, followed by the subtitle 'Register a new manager account for QueueVision'. The form contains four input fields: 'Display Name' with placeholder 'Your name', 'Email' with placeholder 'manager@queuevision.com', 'Password' with placeholder 'Create a strong password' and an eye icon, and 'Confirm Password' with placeholder 'Re-enter your password' and an eye icon. A large blue 'Create account' button is at the bottom, with a link 'Already have an account? Sign in' below it.

Create account

Register a new manager account for QueueVision

Display Name

Your name

Email

manager@queuevision.com

Password

Create a strong password

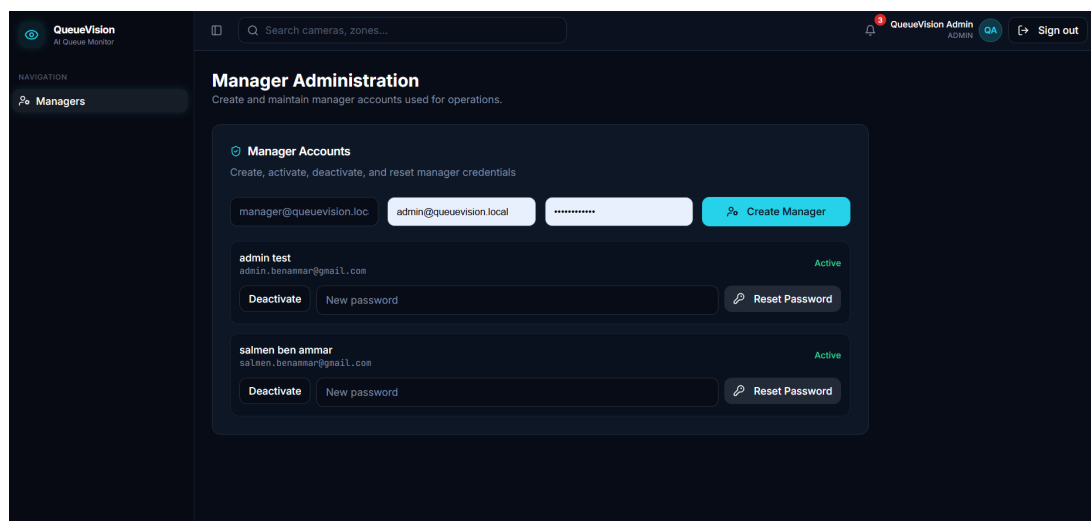
Confirm Password

Re-enter your password

Create account

Already have an account? [Sign in](#)

Figure 5.3: Manager account creation screen.



The image shows the 'Manager Administration' interface. It has a sidebar with 'QueueVision AI Queue Monitor' and 'Managers'. The main content area has a search bar and a 'QueueVision Admin QA Sign out' header. The title 'Manager Administration' is followed by the subtitle 'Create and maintain manager accounts used for operations.' Below this is a 'Manager Accounts' section with the instruction 'Create, activate, deactivate, and reset manager credentials'. It features a 'Create Manager' button and a table of existing accounts. The table has columns for name, email, status, and actions. Two accounts are listed: 'admin test' and 'salmen ben ammar', both with 'Active' status and 'Deactivate' and 'Reset Password' actions.

QueueVision
AI Queue Monitor

Search cameras, zones...

QueueVision Admin QA Sign out

Manager Administration

Create and maintain manager accounts used for operations.

Manager Accounts

Create, activate, deactivate, and reset manager credentials

manager@queuevision.loc admin@queuevision.local Create Manager

admin test
admin.benamar@gmail.com Active

Deactivate New password Reset Password

salmen ben ammar
salmen.benamar@gmail.com Active

Deactivate New password Reset Password

Figure 5.4: Administrator manager-administration interface.

- batched updates to keep the interface responsive.

When the WebSocket connection drops, the dashboard switches to polling and keeps the monitoring surface usable. In this context, continuous visibility is more important than using a single ideal transport. The manager still needs enough information to understand feed state and queue pressure during a temporary network interruption.

5.4.6 Live Video Delivery and Transport Selection

Video transport is selected according to the source type and availability. IP cameras use WebRTC when possible because it gives lower latency for live monitoring. Local MP4 videos use MJPEG, which is simpler and more appropriate for file-based playback.

For IP camera feeds, the browser negotiates a receiving stream with the backend and then transitions to live playback. If the primary transport fails repeatedly, the dashboard stops retrying aggressively and moves to a fallback view. This behavior prevents the interface from depending on one transport mechanism and keeps the feed area meaningful even when WebRTC is unavailable.

5.4.7 Dashboard UI Integration

The dashboard integrates live feed controls, setup dialogs, onboarding, KPI visualizations, and charts. Onboarding supports both uploaded videos and ONVIF sources. For cameras, the manager discovers devices, supplies per-camera credentials, resolves RTSP streams, validates the selection, captures a snapshot for zone definition, and stages the feed for launch. Figure 5.5 shows the operational monitoring surface.

5.4.8 Manager Onboarding Workflow

The manager onboarding flow covers the full path from feed setup to feed confirmation. Figure 5.6 shows the main phases: source selection, source configuration, connection

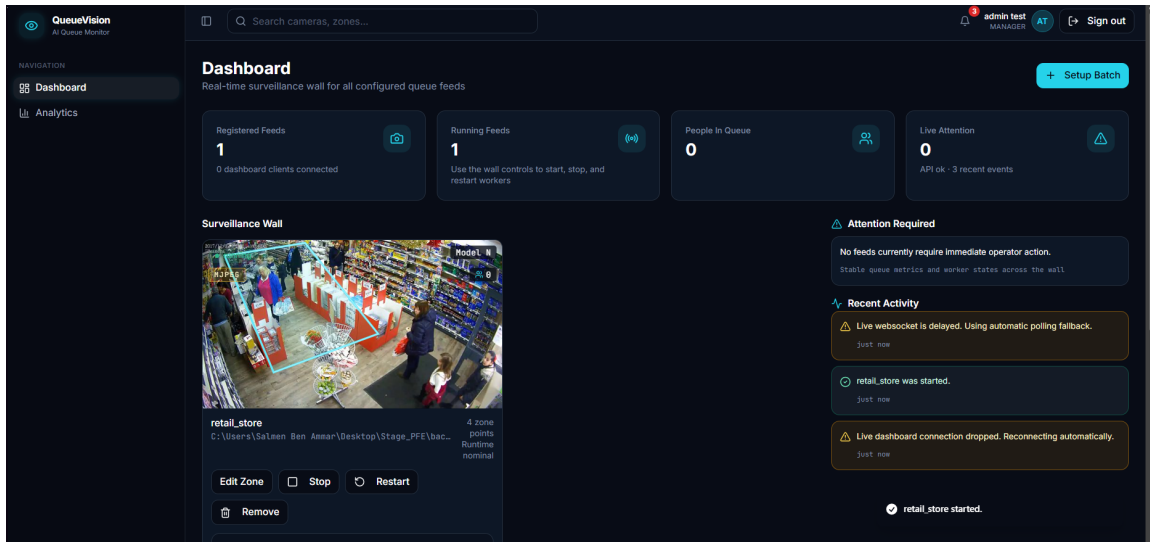


Figure 5.5: Manager dashboard: live surface, feed summaries, and recent activity.

testing, zone definition, model selection, and final review.

The workflow execution involves several screens. First, the feed setup screen prepares an uploaded video or ONVIF source:

Then the zone is defined through the tracing interface:

Model selection precedes the final confirmation. The manager chooses the YOLO size here:

Finally, the staged feed or batch is confirmed before launch:

The practical result is that a manager can move from onboarding to monitoring without leaving the dashboard context. Setup, validation, zone definition, launch, and supervision remain part of the same workflow.

5.4.9 Analytics and Historical Reporting

The analytics view complements live supervision with historical reporting. It aggregates archived alerts and queue metrics so managers can identify repeated congestion patterns and compare queue zones over time.

Four KPI cards summarize average wait time, peak queue length, stability score, and total alerts. A period selector controls hourly, daily, or weekly granularity, while date filters isolate a specific window. The dashboard visualizes wait and queue trends,

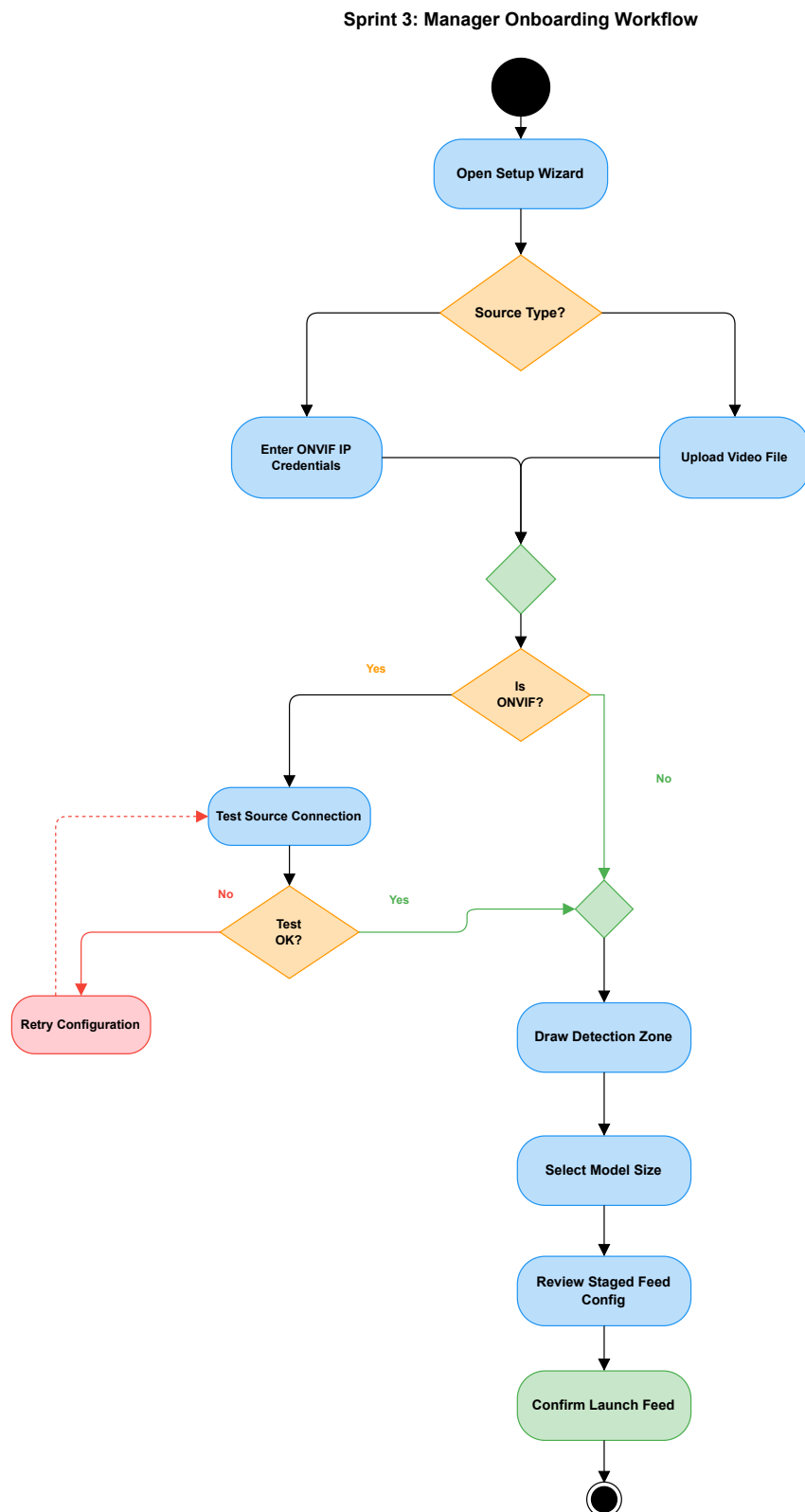


Figure 5.6: Manager onboarding activity diagram showing user actions and UI state transitions from feed setup initialization through feed launch confirmation. Decision nodes show ONVIF versus upload path selection and the test validation retry loop.

Stage Sources ×

Enter the feed information for one source, optionally attach metadata, then continue through zone tracing, model selection, and review.

One source is prepared at a time. Finish this source, then continue to zone tracing and model selection.

Feed name

Checkout 1

Source type

Local MP4 ONVIF Discovery

Upload a local video
Supported formats: MP4, MOV, AVI, and MKV. The backend stores the file and uses its path automatically.

Choose Video File No video selected yet

Local videos are configured one at a time so each feed keeps a single zone and model.

Step 1 uploads the source file into the backend workflow. Step 2 lets you draw the queue polygon on the video frame. Step 3 selects the model size.

Pick one local video to see it listed here.

Cancel Continue to Zone

Figure 5.7: Feed setup screen used to prepare an uploaded video or ONVIF camera source before zone tracing and model selection.

ranks top and bottom zones, and breaks down alerts by severity. CSV export supports external reporting. During early pilots when the archive is empty, the interface uses active-feed data so the analytics page remains useful for demonstration and review.

Figure 5.11 shows the analytics dashboard.

5.4.10 Integration Testing

The dashboard was validated through scenarios covering setup flows, live event handling, transport fallback, feed routing, and error handling.

These tests were useful because the dashboard is not correct just because it renders. It also has to behave properly when live events reconnect, a feed changes state, a route

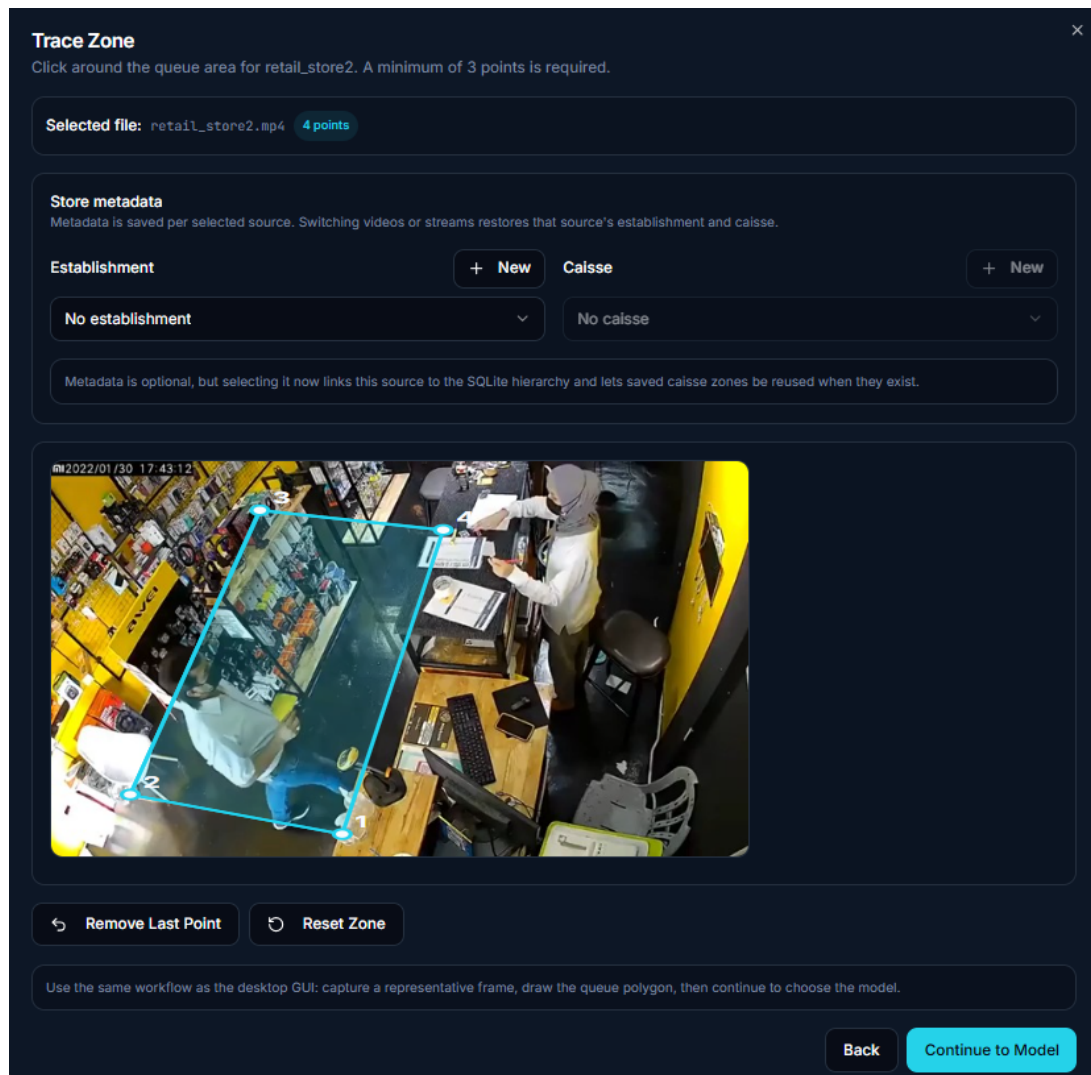


Figure 5.8: Zone tracing screen used to define the queue polygon for the selected feed. is protected, or a fallback transport is activated. The validation therefore focused on the workflows a manager would actually experience during monitoring.

5.5 Conclusion

Sprint 3 delivered the manager-facing monitoring interface and connected it to the live system behavior prepared in Sprint 2. The dashboard now supports authentication, feed setup, ONVIF onboarding, zone selection, model choice, launch review, live metrics, video transport selection, analytics, and administration screens.

The most important result was continuity of use. When live events arrive normally, the dashboard updates without refresh. When WebSocket events are interrupted,

Choose Model

×

Finalize retail_store2 with the YOLO model size you want to run.

SOURCE TYPE	FEED NAME	ZONE POINTS	ESTABLISHMENT
Uploaded video	retail_store2	4	Not assigned
CAISSE			Not assigned

N - Nano
Best for rapid iteration
Fastest setup for quick validation and lightweight machines.

S - Small
Good default for development
Balanced choice when you want better detection quality without a large hit.

M - Medium
Solid for denser queues
More accurate across crowded scenes with a moderate compute budget.

L - Large
Heavier runtime cost
Higher accuracy for harder scenes when your GPU budget allows it.

X - X-Large
Slowest option
Maximum accuracy target for controlled, powerful environments.

BackContinue to Review

Figure 5.9: Model selection screen used to choose the YOLO size before launching the feed.

polling keeps the manager informed. When the preferred video transport is unavailable, fallback behavior keeps the feed area usable. These choices made the interface suitable for the alert automation and final validation work in Sprint 4.

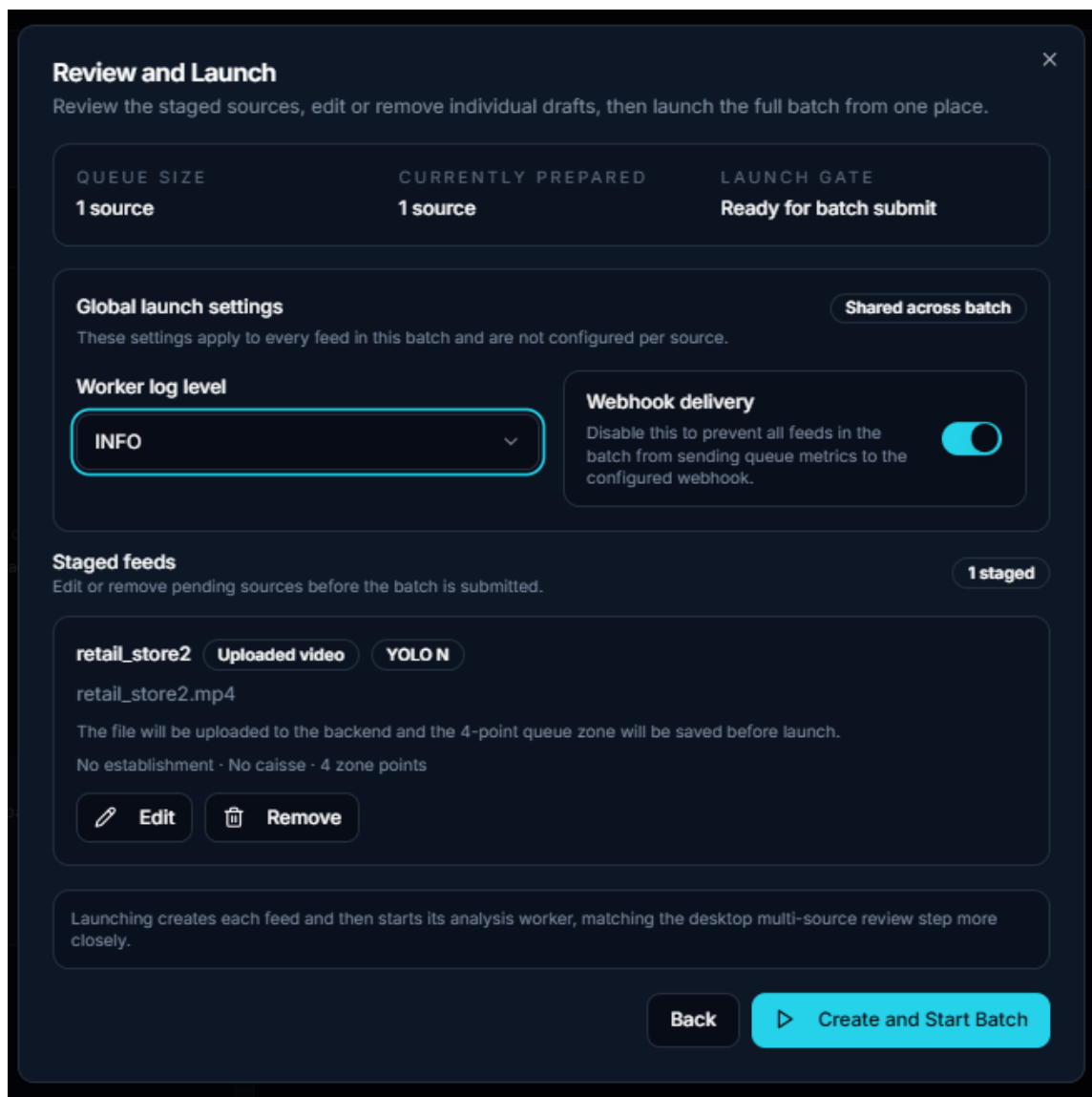


Figure 5.10: Review and launch panel used to confirm staged feeds before batch creation or launch.

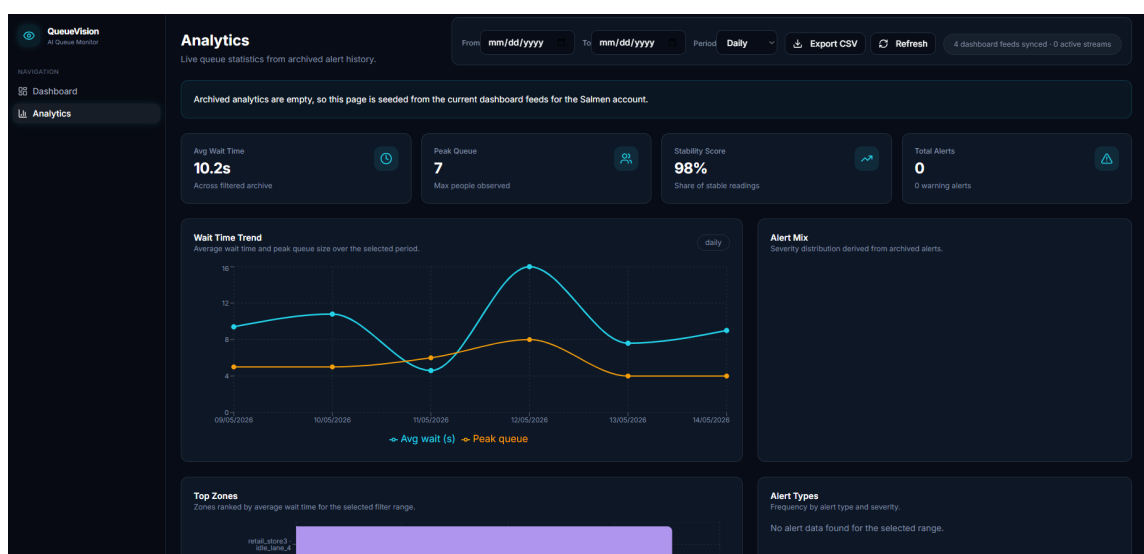


Figure 5.11: Analytics dashboard: KPI summaries, trends, zone rankings, and export.

Sprint 4: Automation, Evaluation, and Optimization

Table of Content

6.1	Introduction	60
6.2	Sprint Objectives and Backlog	60
6.2.1	Technical Tasks	61
6.3	Design	62
6.3.1	Alert Automation and n8n Workflow	62
6.3.2	System Verification and Evaluation Strategy	62
6.4	Implementation	63
6.4.1	Alert Payload and Delivery Architecture	63
6.4.2	Telegram Integration and Notification Delivery	63
6.4.3	Asynchronous Alert Dispatch and Deduplication	65
6.4.4	n8n Workflow Structure and Validation	66
6.4.5	Comprehensive Integration Verification	66
6.4.6	Runtime Optimization and Performance Tuning	67
6.4.7	Fault-Tolerance and Recovery Mechanisms	67
6.4.8	System Integration and Operational Readiness	67
6.5	Conclusion	67

6.1 Introduction

Sprint 4 completed the monitoring loop by connecting queue alerts to an external notification channel and validating the system as a whole. Until this sprint, the project could detect queues, estimate waiting time, expose live services, and display the dashboard. The remaining step was to make alerts reach operational staff and to verify that the full chain behaved correctly under realistic conditions.

The sprint therefore combined two types of work: alert automation through n8n and Telegram, and final evaluation of perception, live monitoring, transport fallback, and notification delivery.

6.2 Sprint Objectives and Backlog

Sprint 4 focused on alert automation and system-wide verification. Alert automation required a validated payload, secure workflow access, cooldown handling, and Telegram delivery. Verification required testing the integrated system, checking streaming stability, tuning runtime responsiveness, and improving recovery from temporary failures. The two main manager needs were alert automation and reliable notification delivery. Table 6.1 lists the user stories that drove this work.

User Story ID	User Story	Task ID	Task
US4.1	As a manager, I want alert payloads dispatched to n8n so that operational staff can be notified automatically.	T4.1	Define the webhook payload schema and validate it before dispatch.
US4.3	As a manager, I want alerts routed through Telegram so that the on-site operator receives notifications on a familiar channel.	T4.3	Create and configure the Telegram bot, store its credential in n8n, and configure the workflow.

Table 6.1: Sprint 4 user stories

6.2.1 Technical Tasks

Beyond the user stories, the sprint included reliability, testing, and performance work.

The technical breakdown is shown in Table 6.2.

Task ID	Task	Subtask ID	Subtask
T4.2	Add retry logic, timeout handling, and logging for webhook delivery to ensure notifications are not silently lost.	ST4.2.1	Implement delivery resilience with exponential backoff and failure tracking.
T4.4	Prepare PowerShell scripts for single-alert, multi-feed, and execution polling scenarios to validate n8n workflow behavior.	ST4.4.1	Create repeatable test automation for workflow routing and alerting paths.
T4.5	Run backend and frontend tests together and confirm lifecycle, streaming, and API contracts under realistic conditions.	ST4.5.1	Execute comprehensive integration verification to catch issues before delivery.
T4.6	Apply runtime tuning: configurable image size, frame stride, async dispatch, and playback pacing to improve processing responsiveness.	ST4.6.1	Implement performance optimizations for the main processing loop.
T4.7	Improve feed worker supervision, frame handoff continuity, and websocket reconnection to strengthen fault tolerance.	ST4.7.1	Enhance system recovery from transient failures and network disruptions.

Table 6.2: Sprint 4 technical tasks

6.3 Design

6.3.1 Alert Automation and n8n Workflow

The alert workflow was designed to avoid two opposite problems: missing important queue alerts and overwhelming staff with repeated notifications for the same condition. When an alert arrives, the workflow first checks that the message is authorized. It then applies cooldown logic, archives the alert in the database, and sends the validated notification to Telegram.

This separation makes the workflow easier to reason about. Authorization protects the entry point, cooldown controls repetition, archival supports traceability, and Telegram delivery reaches the on-site staff channel. Figure 6.1 illustrates the complete workflow logic.

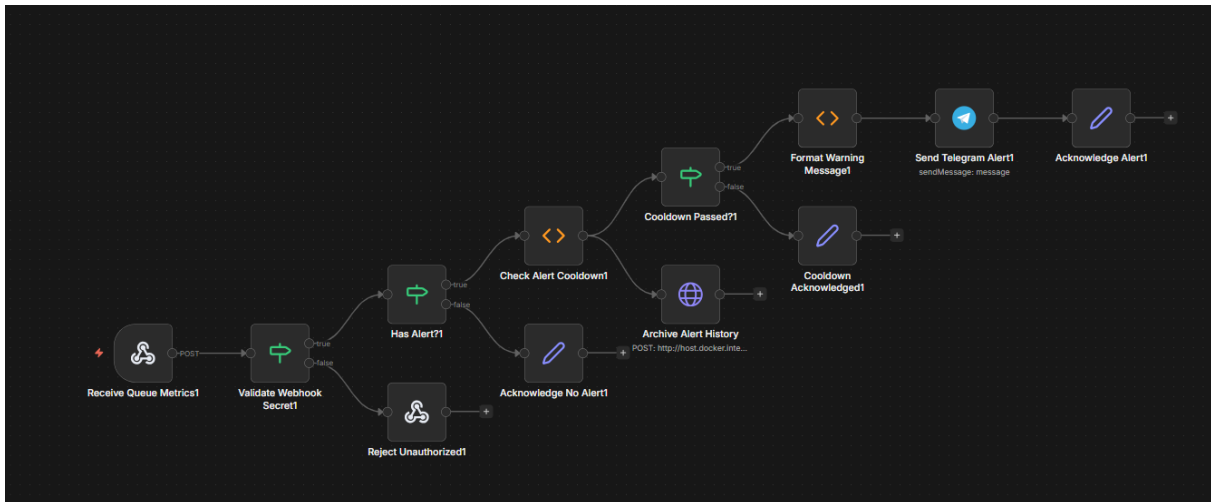


Figure 6.1: Sprint 4 n8n workflow for routing queue alerts through cooldown checks, archival, and Telegram delivery.

6.3.2 System Verification and Evaluation Strategy

The evaluation strategy covered the main parts of the finished system: queue analytics, API behavior, dashboard responsiveness, transport fallback, and n8n routing. The most important observations were whether the perception pipeline stayed responsive,

whether streams recovered after feed changes, whether alerts reached Telegram, and whether the dashboard stayed usable when WebSocket delivery was interrupted.

The tests combined focused checks for individual behaviors with integrated scenarios that followed the system from queue-state change to dashboard update or Telegram notification. Normal operation, temporary failures, feed lifecycle changes, and alert cooldown behavior were all included because these are the situations most likely to affect a real monitoring deployment.

Figure 6.2 illustrates the iterative evaluation and tuning cycle that characterized the sprint.

6.4 Implementation

6.4.1 Alert Payload and Delivery Architecture

The alert payload contains the information needed by the automation workflow and the Telegram message: timestamp and frame context, queue metrics (λ , μ , and expected wait time), alert flags, and severity. Payload validation happens before dispatch so incomplete or malformed messages are not sent to external workflow tools.

Delivery reliability was handled through retry behavior, timeouts, shared-secret validation, and dispatch logging. Successful and failed delivery attempts are kept for traceability, which is important when a manager needs to know whether a threshold violation was actually sent to the notification workflow.

6.4.2 Telegram Integration and Notification Delivery

A Telegram bot was configured as the notification channel, with its credential stored in `n8n` rather than directly inside the exported workflow. This keeps the workflow portable and avoids exposing secrets in shared project artifacts. Alerts that pass authorization

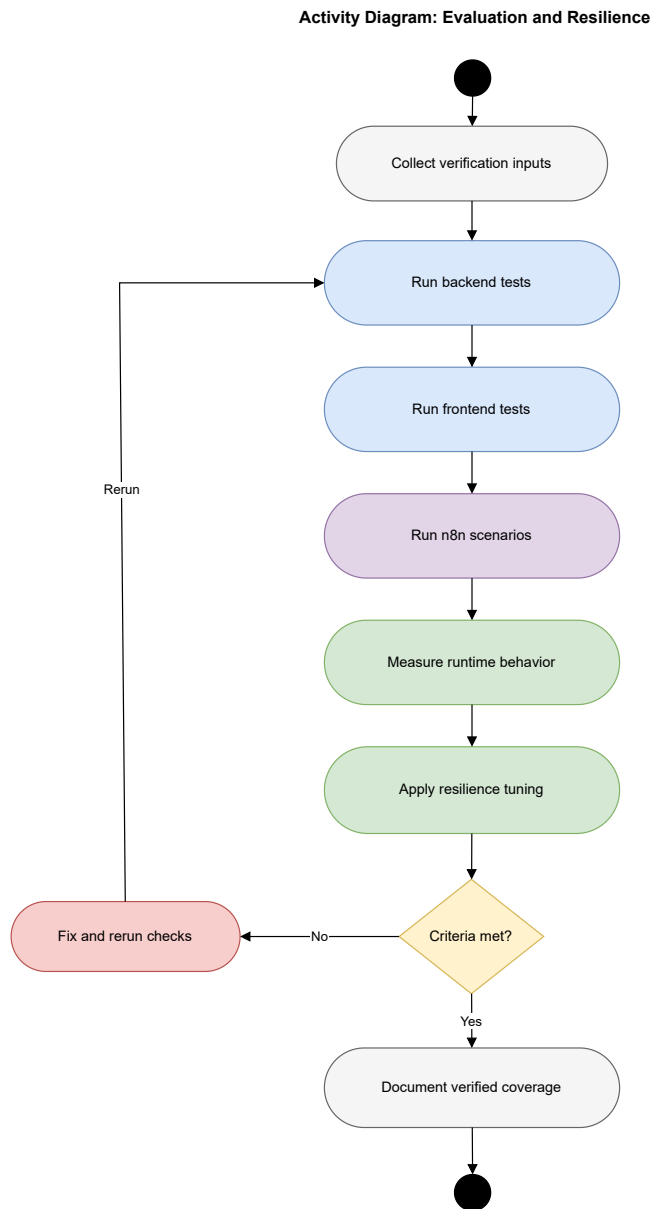


Figure 6.2: Sprint 4 evaluation and resilience activity diagram for tests, tuning, and recovery verification.

and cooldown checks are formatted for human readability and sent to the operational group chat. An example of delivered alert formatting is shown in Figure 6.3.

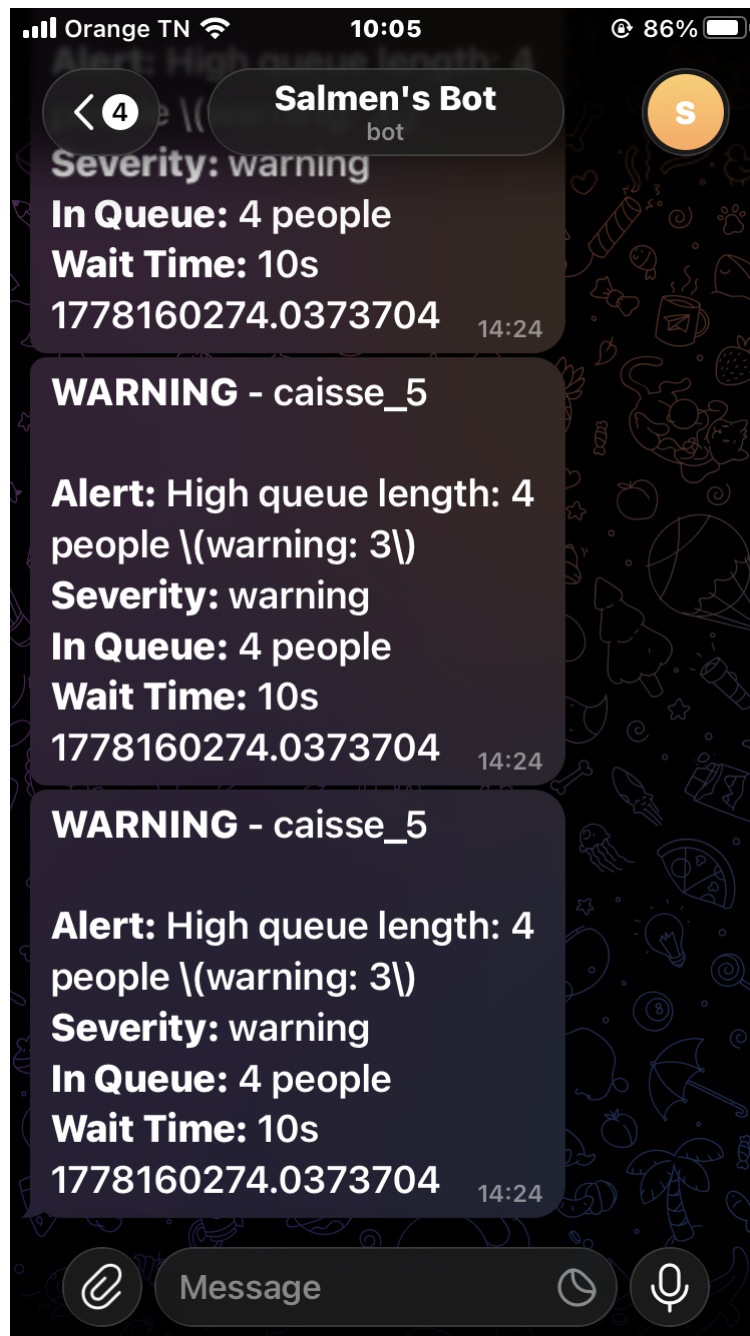


Figure 6.3: Example Telegram alert delivered.

6.4.3 Asynchronous Alert Dispatch and Deduplication

Alert delivery was separated from the perception loop so that network latency would not slow frame processing. This matters because webhook calls depend on external

services and can be delayed or fail temporarily. Queue analysis should continue even if the notification path is slower than expected.

Deduplication reduces repeated notifications when the same threshold violation remains active across several metric windows. A new alert is still sent when the condition changes or a new threshold violation appears, but persistent conditions do not flood the Telegram group.

6.4.4 n8n Workflow Structure and Validation

The n8n workflow separates three alert states: no alert, suppressed alert during cooldown, and delivery-ready alert. This structure made validation clearer because each branch could be exercised independently.

PowerShell validation scripts were used for single-alert scenarios, multi-feed scenarios, and execution polling. These checks confirmed that authorization, cooldown routing, and Telegram delivery behaved as expected across different alert paths.

6.4.5 Comprehensive Integration Verification

Integration verification covered the complete monitoring chain: feed lifecycle actions, streaming behavior, live dashboard updates, webhook delivery, and Telegram notification routing. Error scenarios included authentication failures, WebSocket disconnections, WebRTC unavailability, and feed worker interruption.

The goal was not only to confirm that each feature worked in isolation, but also to check that failures in one part of the system did not make the whole platform unusable. For example, a WebSocket interruption should move the dashboard to polling, and a temporary notification failure should be recorded rather than silently ignored.

6.4.6 Runtime Optimization and Performance Tuning

Several runtime tuning options were added to improve responsiveness. Image size and frame stride can be adjusted so the project can trade some detection detail for faster processing when hardware resources are limited. Device selection supports different deployment machines. Alert dispatch was moved away from the main perception path, and file playback pacing was adjusted to better simulate real-time delivery instead of pushing frames as fast as possible.

6.4.7 Fault-Tolerance and Recovery Mechanisms

Fault tolerance was improved at several points in the system. Feed supervision helps recover from worker interruption, frame handoff was hardened to reduce loss during transitions, and the dashboard reconnects automatically before falling back to polling. Together, these behaviors keep monitoring available during temporary failures rather than requiring a full manual restart for every interruption.

6.4.8 System Integration and Operational Readiness

By the end of Sprint 4, the perception pipeline, queue analytics, backend services, web dashboard, and n8n alert automation were connected into one working platform. The remaining work before production deployment is mainly deeper long-duration testing, load evaluation, and infrastructure hardening, not the completion of missing core features.

6.5 Conclusion

Sprint 4 delivered the final monitoring-to-action loop. Queue alerts could be validated, routed through n8n, filtered by cooldown logic, and delivered to Telegram. The evalua-

tion work confirmed the main integrated behaviors: live metrics, dashboard resilience, stream fallback, alert dispatch, and workflow routing.

The sprint also showed why end-to-end validation is necessary for this type of project. Component checks can confirm isolated behavior, but only integrated scenarios reveal whether the manager still receives useful information when transport, notification, or feed conditions degrade. After this sprint, the project reached a functional level suitable for demonstration and further production-oriented validation.

General Conclusion

Table of Content

7.1	Summary	70
7.2	Objective Review	71
7.3	Future Work	71

7.1 Summary

This end-of-studies project produced a queue wait-time estimation platform that connects computer vision, queue analytics, web supervision, and alert automation. The project started from a practical problem: managers need more than a visual impression of a queue. They need indicators that show how the queue is evolving and when action is required.

In practical terms, the system now includes:

- a perception pipeline based on YOLO detection and ByteTrack identity continuity,
- queue-zone filtering and event-driven estimation of λ , μ , and expected wait time,
- adaptive wait-time outputs that respond to queue conditions,
- a secure authentication layer for manager sign-up, sign-in, session refresh, and administrator account governance,
- a typed FastAPI backend-for-frontend with feed lifecycle control and live web-socket events,
- a React dashboard for monitoring, setup, and runtime interaction,
- n8n and Telegram integration for operational alert routing.

Taken together, these elements form a monitoring-to-action loop: the system observes a queue, estimates its state, displays it to the manager, and sends alerts when the situation requires attention.

Across the four sprint chapters, the work progressed from perception and queue analytics to live services, then to the web dashboard, and finally to automation and validation.

7.2 Objective Review

The initial objectives were achieved at the project scope level:

1. **Real-time detection and tracking:** achieved through YOLO-based detection and ByteTrack identity continuity.
2. **Queue-zone counting:** achieved with polygon-zone management and in-zone identity filtering.
3. **Queue metric estimation:** achieved for arrival rate, service rate, and wait time.
4. **Adaptive wait-time estimation:** achieved through automatic formula switching based on queue load.
5. **Web monitoring:** achieved through a multi-feed dashboard with live updates, setup workflows, and fallback behavior.
6. **Automation integration:** achieved through webhook transport, n8n routing, and Telegram notifications.

The main remaining gap is not architectural completeness, but the depth of long-duration evaluation under broader operational scenarios.

7.3 Future Work

Several extensions would make the platform stronger before wider deployment:

- multi-zone and multi-server queue modeling for parallel cashier configurations,
- richer alert strategies with cooldown tuning, escalation policies, and acknowledgment loops,
- edge deployment optimization through ONNX/TensorRT profiling and resource-aware scheduling,

GENERAL CONCLUSION

- long-duration field evaluation with domain-specific KPI baselines,
- stronger security controls for production webhook exposure and secrets management.



References

- [1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Wiley, 2018.
- [2] Ultralytics, *Ultralytics documentation – yolo real-time object detection*, <https://docs.ultralytics.com/>, Accessed: 2026-03-01, 2026.
- [3] Waitwhile, *Waitwhile – virtual queue and appointment management*, <https://www.waitwhile.com/>, Accessed: 2026-02-24, 2026.
- [4] Qminder, *Qminder – queue management system*, <https://www.qminder.com/>, Accessed: 2026-02-24, 2026.
- [5] RetailNext, *Retailnext – in-store analytics and traffic intelligence*, <https://retailnext.net/>, Accessed: 2026-02-25, 2026.
- [6] Xovis, *Xovis – 3d sensor-based people flow and queue measurement*, <https://www.xovis.com/>, Accessed: 2026-02-25, 2026.
- [7] Roboflow, *Roboflow – computer vision platform*, <https://roboflow.com/>, Accessed: 2026-02-26, 2026.
- [8] Microsoft, *Visual studio code documentation*, <https://code.visualstudio.com/docs>, Accessed: 2026-02-21, 2026.
- [9] Git, *Git documentation*, <https://git-scm.com/doc>, Accessed: 2026-02-20, 2026.
- [10] GitHub, *Github documentation*, <https://docs.github.com/>, Accessed: 2026-02-20, 2026.
- [11] diagrams.net, *Diagrams.net (draw.io) documentation*, <https://www.diagrams.net/>, Accessed: 2026-02-23, 2026.

REFERENCES

- [12] Docker, *Docker documentation – containers and compose*, <https://docs.docker.com/>, Accessed: 2026-02-19, 2026.
- [13] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” *arXiv preprint arXiv:2110.06864*, 2022.
- [14] OpenCV, *OpenCV documentation*, <https://docs.opencv.org/>, Accessed: 2026-02-27, 2026.
- [15] NumPy, *Numpy documentation*, <https://numpy.org/doc/>, Accessed: 2026-02-25, 2026.
- [16] pandas, *Pandas documentation*, <https://pandas.pydata.org/docs/>, Accessed: 2026-04-01, 2026.
- [17] SciPy, *Scipy documentation*, <https://docs.scipy.org/doc/scipy/>, Accessed: 2026-04-03, 2026.
- [18] S. Ramírez, *Fastapi documentation*, <https://fastapi.tiangolo.com/>, Accessed: 2026-03-17, 2026.
- [19] Pydantic, *Pydantic documentation*, <https://docs.pydantic.dev/>, Accessed: 2026-03-18, 2026.
- [20] W3C Web Real-Time Communications Working Group, *WebRTC 1.0: Real-time communication between browsers*, <https://www.w3.org/TR/webrtc/>, Accessed: 2026-03-27, 2025.
- [21] MediaMTX, *Mediamtx – media server documentation*, <https://github.com/bluenvoron/mediamtx>, Accessed: 2026-03-24, 2026.
- [22] React, *React documentation*, <https://react.dev/>, Accessed: 2026-04-14, 2026.
- [23] Microsoft, *Typescript documentation*, <https://www.typescriptlang.org/docs/>, Accessed: 2026-04-15, 2026.
- [24] Vite, *Vite documentation*, <https://vite.dev/guide/>, Accessed: 2026-04-16, 2026.

REFERENCES

- [25] T. CSS, *Tailwind css documentation*, <https://tailwindcss.com/docs/>, Accessed: 2026-04-18, 2026.
- [26] n8n, *N8n documentation – workflow automation*, <https://docs.n8n.io/>, Accessed: 2026-05-13, 2026.
- [27] Telegram, *Telegram bot api*, <https://core.telegram.org/bots/api>, Accessed: 2026-05-15, 2026.
- [28] SQLite, *Sqlite documentation*, <https://www.sqlite.org/docs.html>, Accessed: 2026-03-20, 2026.
- [29] Internet Engineering Task Force, *Rfc 2326: Real time streaming protocol (rtsp)*, <https://www.rfc-editor.org/rfc/rfc2326>, Accessed: 2026-03-08, 1998.
- [30] ONVIF, *Onvif specifications*, <https://www.onvif.org/profiles/specifications/>, Accessed: 2026-03-06, 2026.